

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ПО ПРОГРАММЕ БАКАЛАВРИАТА**

09.03.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Разработка программно-информационных систем»

Разработка программной системы компилятора функционального
языка программирования Common Lisp с оптимизациями

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

(подпись, дата)

Д. С. Шатохин

(инициалы, фамилия)

Группа ПО-216

Руководитель ВКР

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

(подпись, дата)

А. В. Малышев

(инициалы, фамилия)

Курск 2026 г.

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

УТВЕРЖДАЮ:

Заведующий кафедрой

(подпись, инициалы, фамилия)

«____» _____ 20____ г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ БАКАЛАВРИАТА

Студента Шатохина Д. С., шифр 22-06-0118, группа ПО-216

1. Тема «Разработка программной системы компилятора функционального языка программирования Common Lisp с оптимизациями» утверждена приказом ректора ЮЗГУ от «03» апреля 2026 г. № 1584-с.

2. Срок предоставления работы к защите «9» июня 2026 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

1) изучение процесса компиляции языков программирования и функциональных языков в частности;

2) изучение методов оптимизации компиляторов;

3) проектирование компилятора с оптимизациями;

4) реализация компиляции S-выражения в промежуточную форму;

5) реализация генерации кода ассемблера;

6) реализация генерации байт-кода;

7) реализация оптимизации промежуточной формы.

3.2. Входные данные и требуемые результаты для программы:

1) Входными данными для программной системы является исходный код программы на языке Common Lisp.

2) Выходными данными для программной системы является байт-код для выполнения исходной программы на виртуальной машине.

4. Содержание работы (по разделам):

4.1. Введение.

4.1. Анализ предметной области.

4.2. Техническое задание: основание для разработки, назначение разработки, требования к программной системе, требования к оформлению документации.

4.3. Технический проект: общие сведения о программной системе, проект данных программной системы, проектирование архитектуры программной системы, проектирование программного интерфейса системы.

4.4. Рабочий проект: спецификация компонентов программной системы, тестирование программной системы, сборка компонентов программной системы.

4.5. Заключение.

4.6. Список использованных источников.

5. Перечень графического материала:

Лист 1. Сведения о ВКРБ.

Лист 2. Цель и задачи разработки.

Лист 3. Язык Common Lisp.

Лист 4. Архитектура компилятора.

Лист 5. Фаза компиляции.

Лист 6. Фаза оптимизации.

Лист 7. Фаза генерации кода.

Лист 8. Фаза ассемблирования.

Лист 9. Архитектура виртуальной машины.

Лист 10. Тестирование.

Лист 11. Заключение.

Руководитель ВКР

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

Задание принял к исполнению

(подпись, дата)

Д. С. Шатохин

(инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 129 страницам. Работа содержит 60 иллюстраций, 14 таблиц, 56 библиографических источников и 11 листов графического материала. Количество приложений – 2. Графический материал представлен в приложении А. Фрагменты исходного кода представлены в приложении Б.

Перечень ключевых слов: компилятор, транслятор, лексер, парсер, код, ассемблер, виртуальная машина, байт-код, инструкция, регистр, кадр активации, стек, функциональное программирование, Common Lisp, S-выражение, макросы, рекурсия, оптимизация.

Объектом разработки является программная система компилятора языка программирования Common Lisp.

Целью выпускной квалификационной работы является создание программной системы компилятора языка программирования Common Lisp.

В ходе работы были выделены основные этапы компиляции и реализованы соответствующие модули: преобразование исходного S-выражения в промежуточную форму с упрощёнными операциями, оптимизация этой формы, генерация кода ассемблера и трансляция в байт-код виртуальной машины. Оптимизатор проходит промежуточное дерево в глубину.

Виртуальная машина имеет следующие регистры: программный счётчик для последовательного выполнения инструкций и возможности перехода, аккумулятор для промежуточных значений, указатели вершины стека общего назначения, текущего кадра окружения и вершины стека нелокальных переходов. Константы и глобальные переменные хранятся в массивах и загружаются в аккумулятор по индексу. Перед вызовом функции создаётся новый кадр активации, наследующий окружение объявления функция, и в этот кадр активации заносятся значения аргументов через стек.

При разработке компилятора использовался язык программирования Common Lisp.

Разработанный компилятор был успешно внедрен в os-ri.

ABSTRACT

The volume of work is 129 pages. The work contains 60 illustrations, 14 tables, 56 bibliographic sources and 11 sheets of graphic material. The number of applications is 2. The graphic material is presented in annex A. The source code fragments are presented in annex B.

List of keywords: compiler, translator, lexer, parser, code, assembler, virtual machine, bytecode, instruction, register, activation frame, stack, functional programming, Common Lisp, S-expression, macros, recursion, optimization.

The object of the development is a software system for a compiler of the Common Lisp programming language.

The purpose of the final qualifying work is the creation of a software system for a compiler of the Common Lisp programming language.

In the course of the work, the main stages of compilation were identified and the corresponding modules were implemented: conversion of the source S-expression into an intermediate form with simplified operations, optimization of this form, generation of assembler code, and translation into virtual machine bytecode. The optimizer traverses the intermediate tree depth-first.

The virtual machine has the following registers: a program counter for sequential instruction execution and branching capability, an accumulator for intermediate values, a general-purpose stack top pointer, a current environment frame pointer, and a non-local jump stack top pointer. Constants and global variables are stored in arrays and loaded into the accumulator by index. Before a function call, a new activation frame is created, inheriting the function's declaration environment, and argument values are placed into this activation frame via the stack.

The Common Lisp programming language was used in the development of the compiler.

The developed compiler was successfully deployed in os-pi.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	13
1 Анализ предметной области	15
1.1 Введение в компиляторы	15
1.2 История компиляторов	16
1.3 Функциональные языки программирования и Lisp	17
1.4 Компиляция функциональных языков и Lisp	19
1.5 История развития методов оптимизации компиляторов	20
1.6 Основные концепции оптимизации компиляторов	21
1.7 Классификация методов оптимизации	22
1.8 Обзор основных методов оптимизации	23
1.9 Оптимизации компиляции функциональных языков и Lisp	24
2 Техническое задание	28
2.1 Основание для разработки	28
2.2 Цель и назначение разработки	28
2.3 Реализуемое подмножество Common Lisp	28
2.3.1 S-выражения	28
2.3.2 Типы объектов	29
2.3.3 Специальные формы	29
2.3.4 Примитивы	30
2.3.4.1 Списки	30
2.3.4.2 Арифметические операции	30
2.3.4.3 Предикаты	30
2.3.4.4 Побитовые операции	31
2.3.4.5 Строки и символы	31
2.3.4.6 Массивы	31
2.3.4.7 Печать объектов	32
2.3.4.8 Другие примитивы	32
2.3.5 Лямбда выражения	32
2.3.6 Определение функций	32

2.3.7	Глобальные переменные	33
2.3.8	Макросы	33
2.3.9	Функции как объекты первого класса	34
2.3.10	Обработка ошибок	35
2.4	Программный интерфейс оптимизирующего компилятора	36
2.4.1	Флаги оптимизации компилятора	36
2.4.1.1	trivial-condition	37
2.4.1.2	simplify-arithmetic	37
2.4.1.3	dead-code-elimination	37
2.4.1.4	tail-call	38
2.4.1.5	constant-folding	38
2.4.1.6	unused-functions	38
2.4.1.7	beta-expansion	39
2.4.1.8	all-inline	39
2.5	Требования к оформлению документации	40
3	Технический проект	41
3.1	Общая характеристика организации решения задачи	41
3.2	Архитектура оптимизирующего компилятора Common Lisp	41
3.3	Фаза 1. Макроподстановка	43
3.3.1	Атомарные выражения и окружение макросов	43
3.3.2	Вызов примитива	44
3.3.3	Пользовательские функции	45
3.3.4	if	45
3.3.5	setq	46
3.3.6	progn	46
3.3.7	let	47
3.3.8	cond	47
3.3.9	quote	47
3.3.10	backquote	48
3.3.11	function	48
3.3.12	funcall	49

3.4	Фаза 2. Статический анализ	49
3.4.1	Макросы	49
3.4.2	Лексическое окружение	49
3.4.3	Константы и переменные	50
3.4.4	progn	51
3.4.5	if	51
3.4.6	setq	52
3.4.7	Функции и замыкания	52
3.4.7.1	tagbody	54
3.4.8	catch/throw	54
3.4.9	Операции после первой фазы	54
3.5	Фаза 3. Оптимизация	57
3.5.1	Оптимизация тривиальных условий (trivial-condition)	57
3.5.2	Оптимизация арифметических выражений (simplify-arithmetic)	57
3.5.3	Удаление неиспользуемого кода (dead-code-elimination)	58
3.5.3.1	Оптимизация загрузки регистра аккумулятора	58
3.5.3.2	Удаление функций после подстановки	59
3.5.4	Оптимизация хвостовой рекурсии (tail-call)	59
3.5.5	Свёртка констант (constant-folding)	59
3.5.6	Удаление неиспользуемых функций (unused-functions)	60
3.5.7	Встраивание функций, которые вызываются один раз (beta-expansion)	60
3.5.8	Полное встраивание функций (all-inline)	61
3.6	Фаза 4. Генерация линейных инструкций	61
3.7	Фаза 5. Ассемблер	64
3.8	Архитектура целевой виртуальной машины	65
4	Рабочий проект	73
4.1	common.lsp	73
4.2	macro.lsp	74
4.3	compiler.lsp	76
4.4	optimizer.lsp	79

4.5	generator.lsp	81
4.6	assembler.lsp	83
4.7	Модульное тестирование фазы компиляции	84
4.8	Модульное тестирование фазы оптимизации	85
4.9	Системное тестирование разработанного компилятора	87
4.9.1	Технология системного тестирования	87
4.9.2	Тестирование «AES»	89
4.9.3	Тестирование «regex»	90
4.9.4	Тестирование «PNG»	91
4.9.5	Тестирование «JPEG»	91
4.9.6	Тестирование «Lua»	92
	ЗАКЛЮЧЕНИЕ	94
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	95
	ПРИЛОЖЕНИЕ А Представление графического материала	102
	ПРИЛОЖЕНИЕ Б Фрагменты исходного кода программы	114
	На отдельных листах (CD-RW в прикрепленном конверте)	129
	Сведения о ВКРБ (Графический материал / Сведения о ВКРБ.png)	Лист 1
	Цель и задачи разработки (Графический материал / Цель и задачи разработки.png)	Лист 2
	Язык Common Lisp (Графический материал / Язык Common Lisp.png)	Лист 3
	Архитектура компилятора (Графический материал / Архитектура компилятора.png)	Лист 4
	Фаза компиляции (Графический материал / Фаза компиляции.png)	Лист 5
	Фаза оптимизации (Графический материал / Фаза оптимизации.png)	Лист 6
	Фаза генерации кода (Графический материал / Фаза генерации кода.png)	Лист 7
	Фаза ассемблирования (Графический материал / Фаза ассемблирования.png)	Лист 8
	Архитектура виртуальной машины (Графический материал / Архитектура виртуальной машины.png)	Лист 9
	Тестирование (Графический материал / Тестирование.png)	Лист 10

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

ИС – информационная система.

ИТ – информационные технологии.

ПО – программное обеспечение.

ЯП – язык программирования.

AST – abstract syntax tree; абстрактное синтаксическое дерево.

CL – язык программирования Common Lisp.

GC – garbage collector; сборщик мусора.

ЛТ – just-in-time; метод трансляции и исполнения программы во время её работы.

SSA – static single assignment; форма статического однократного присваивания в промежуточном представлении программы.

ТЗ – техническое задание.

ТП – технический проект.

РП – рабочий проект.

ВВЕДЕНИЕ

Компилятор языка программирования Common Lisp представляет собой программную систему, осуществляющую трансляцию исходного кода, написанного на языке высокого уровня, в машинный код или байт-код виртуальной машины. В отличие от интерпретаторов, компилятор выполняет статический анализ и преобразование программы перед её исполнением, что открывает возможности для применения глубоких оптимизаций, повышающих эффективность выполнения.

Разработка собственного компилятора с поддержкой современных методов оптимизации позволяет не только достичь высокой производительности генерируемого кода, но и обеспечить полный контроль над процессом трансляции, адаптацию под конкретную среду исполнения, а также возможность исследования и внедрения эффективных приёмов реализации функциональных конструкций, таких как хвостовая рекурсия, замыкания и макросы.

Разработанный компилятор генерирует на основе исходного кода Common Lisp байт-код для выполнения на виртуальной машине. Поддерживаются следующие оптимизации: свёртка констант, удаление неиспользуемого кода, оптимизация хвостовой рекурсии, упрощение тривиальных условий, оптимизация арифметических выражений, встраивание функций.

Цель настоящей работы – реализация программной системы оптимизирующего компилятора Common Lisp. Для достижения поставленной цели необходимо решить *следующие задачи*:

- изучение процесса компиляции языков программирования и функциональных языков в частности;
- изучение методов оптимизации компиляторов;
- проектирование компилятора с оптимизациями;
- реализация компиляции S-выражения в промежуточную форму;
- реализация генерации кода ассемблера;
- реализация генерации байт-кода;
- реализация оптимизации промежуточной формы.

Структура и объем работы. Отчет состоит из введения, 4 разделов основной части, заключения, списка использованных источников, 2 приложений. Текст выпускной квалификационной работы равен 129 страницам.

Во введении сформулирована цель работы, поставлены задачи разработки, описана структура работы, приведено краткое содержание каждого из разделов.

В первом разделе на стадии описания технической характеристики предметной области приводится сбор информации о методах разработки компиляторов.

Во втором разделе на стадии технического задания приводятся требования к разрабатываемому компилятору.

В третьем разделе на стадии технического проектирования представлены проектные решения для компилятора.

В четвертом разделе приводится список модулей и их функций, использованных при разработке компилятора, производится тестирование разработанной программной системы компилятора.

В заключении излагаются основные результаты работы, полученные в ходе разработки.

В приложении А представлен графический материал. В приложении Б представлены фрагменты исходного кода.

1 Анализ предметной области

1.1 Введение в компиляторы

Компилятор представляет собой программу, которая переводит исходный код на одном языке программирования (как правило, высокого уровня) в эквивалентный код на другом языке (чаще всего в машинный код или байт-код).

Процесс компиляции состоит из двух основных этапов: трансляция и компоновка.

Трансляция – это основная часть компиляции, которая включает в себя анализ и преобразование исходного кода в промежуточный вид и затем в машинный код или байт-код. Трансляция программы состоит из нескольких этапов:

1. Лексический анализ. Последовательность символов исходного кода преобразуется в последовательность минимальных смысловых единиц – лексем.

2. Синтаксический анализ. Последовательность лексем преобразуется в синтаксическое дерево (AST), отражающее грамматическую структуру программы в соответствии с правилами языка. На этом этапе выявляются синтаксические ошибки.

3. Семантический анализ. Происходит разбор синтаксического дерева с целью установления его семантики (смысла), например, проверка области видимости переменных и функций, соответствие типов данных и т.д. Результатом данной фазы является более удобное для дальнейшей обработки промежуточное представление.

4. Оптимизация. Выполняется устранение избыточных вычислений и упрощение кода с сохранением его смысла. Оптимизация происходит на разных этапах компиляции – например, над промежуточным кодом или над конечным машинным кодом.

5. Генерация кода. Из промежуточного представления выполняется переход в целевой код (машинный код для конкретной архитектуры процессора

и операционной системы, байт-код для виртуальной машины или код на другом языке программирования). На этом этапе имеют место быть низкоуровневые оптимизации для целевой архитектуры.

Компоновка – процесс объединения отдельных программных модулей, полученных в процессе трансляции, в единый исполняемый файл или библиотеку. Это финальная стадия компиляции, в состав которой входит разрешение внешних ссылок между модулями, организация адресного пространства, а также оптимизации, например удаление неиспользуемого кода.

1.2 История компиляторов

В начале компьютерной эпохи программы писали на машинном языке или на ассемблере, учитывая архитектуру целевого процессора. Это был неудобный и трудоёмкий процесс, из-за чего появилась необходимость в инструментах для облегчения программирования.

Первым компилятором принято считать программу A-0, созданную Грейс Хоппер в начале 1950-х для компьютера UNIVAC I. A-0 позволял писать программы в виде псевдокодов, которые автоматически преобразовывались в машинные инструкции.

Также в этот период появился компилятор для языка FORTRAN, разработанный Джоном Бэкусом и его командой в IBM. FORTRAN стал одним из первых высокоуровневых языков программирования, а его компилятор – первым широко используемым производственным компилятором.

В 1980-х разработчики компиляторов стали уделять большое внимание оптимизации целевого кода. Появились техники для увеличения скорости и уменьшения размера исполняемых программ. Также появилась потребность в портируемости программ, что привело к развитию стандартов языков и созданию промежуточных представлений (например, байт-код Java).

Современные компиляторы обладают сложной структурой и поддерживают множество языков и платформ. Развитие технологий привело к появлению компиляторов с многоуровневыми оптимизациями, динамической (Just-In-Time, JIT) компиляции – компиляция байт-кода в машинный код во

время выполнения (например JVM, CLR), проектов с открытым исходным кодом (open-source), таких как GCC и LLVM, которые предоставляют гибкие и расширяемые платформы для создания компиляторов. Кроме того, компиляторы стали интегрированы в современные среды разработки (IDE), обеспечивая не только трансляцию, но и статический анализ, автодополнение, рефакторинг и отладку кода.

1.3 Функциональные языки программирования и Lisp

Функциональное программирование – парадигма программирования, которая описывает процесс выполнения программы как вычисление значений функций в математическом понимании, в отличие от императивного программирования, в котором выполнение программы определяется последовательным изменением состояний. Функциональные языки программирования характеризуются отсутствием побочных эффектов функций, выражением программ в виде композиции функций и выражений, использованием функций высшего порядка, неизменяемостью данных и ленивыми вычислениями.

Lisp – семейство языков программирования, программы и данные в которых представляются системами односвязных списков из элементов любой природы. Это один из старейших (наряду с Fortran и Cobol) до сих пор используемых языков программирования высокого уровня, а также первый из сохранившихся в использовании языков, использующих автоматическое управление памятью и сборку мусора.

Лисп был создан Джоном Маккарти для работ по искусственному интеллекту и до сих пор остается одним из основных инструментальных средств в данной области. Применяется он и как средство обычного промышленного программирования, от встроенных скриптов до веб-приложений массового использования, хотя популярным его назвать нельзя.

Common Lisp – диалект языка программирования Лисп, стандартизированный ANSI. Стандарт фиксирует язык как мультипарадигменный: поддерживается комбинация процедурного, функционального и

объектно-ориентированного программирования. В частности, объектно-ориентированное программирование обеспечивается входящей в язык системой CLOS, а система лисп-макросов позволяет вводить в язык новые синтаксические конструкции, использовать техники метапрограммирования и обобщенного программирования.

Common Lisp отличается от таких языков, как C#, Java, Perl, Python тем, что он определяется своим стандартом и не существует его единственной или канонической реализации. Любой желающий может ознакомиться со стандартом и создать свою собственную реализацию. Common Lisp автоматически признает все реализации как равные.

Примеры реализаций Common Lisp:

1. CLISP. Компилируется в байт-код, также возможна JIT-компиляция. Маленький размер образа лисп-системы. Очень эффективная длинная целочисленная арифметика. Возможность создания исполняемых файлов. FFI (интерфейс для вызова низкоуровневых функций (функций из библиотек, написанных на Си и т. п.) и для оперирования «неуправляемой» памятью). Функции обратного вызова (интеграция с «родным» кодом платформы).

2. CMUCL. Высококачественный компилятор в машинный код. FFI. Функции обратного вызова (интеграция с «родным» кодом платформы).

3. Clozure CL. Компилируется в машинный код. Быстрый компилятор. Мощный и удобный FFI. Функции обратного вызова (интеграция с бинарным кодом платформы). Возможность создания исполняемых файлов. Многопоточность на всех поддерживаемых платформах.

4. Steel Bank Common Lisp (SBCL). Компилируется в быстрый машинный код с акцентом на производительность. Мощный и стабильный FFI для вызова функций из Си и работы с памятью. Поддержка функций обратного вызова и создание исполняемых файлов. Эффективная многопоточность на большинстве платформ. Активно развивается и широко используется в промышленности.

1.4 Компиляция функциональных языков и Lisp

Функциональные языки программирования традиционно ассоциируются с богатым математическим и абстрактным подходом к обработке данных. В связи с этим у них сложилась особая модель компиляции, отличающаяся от классической императивной. Функции, выражения, лямбда-выражения, которые требуют особых методов обработки при транслировании в машинный код или байт-код. Множество оптимизаций зависит от оценки выражений и замены их на более эффективные аналоги. Мета-программирование и макросистемы позволяют расширять язык, что влияет на стратегии компиляции и обработки кода.

Большинство ранних функциональных языков (например, исходные реализации Lisp в 1950–60-е) работают как интерпретаторы, выполняя код непосредственно из структур данных. Современные реализации, такие как OCaml, Haskell (GHC), Scheme (к примеру, Racket), используют компиляторы, преобразующие исходные программы в байт-код для виртуальных машин. Это обеспечивает хорошую скорость выполнения и интерактивность. Компиляция в машинный код при необходимости достигается за счёт использования специальных компиляторов (например, GHC для Haskell или SBCL для Common Lisp), которые выполняют генерацию нативного кода. В этом случае возможна высокая скорость исполнения. Ленивая (отложенная) оценка – характерная черта многих функциональных языков, особенно Haskell, которая затрудняет традиционные методы компиляции, так как результат вычислений не всегда известен заранее.

Lisp, будучи языком с очень гибкой системой макросов и динамической типизацией, имеет свои уникальные особенности:

1. Интерактивное выполнение и REPL. Большинство реализаций Lisp поддерживают интерактивное программирование, где пользователь может писать код, компилировать его и запускать в реальном времени в режиме «обратной связи». Это обеспечивает быструю итерацию разработки.

2. Компиляция в байт-код и машинный код. Современные реализации Lisp позволяют компилировать программы как в байт-код (например, SBCL), так и в нативный машинный код, что значительно повышает производительность.

3. Макросы и трансформация кода. Макросы Lisp функционируют на уровне синтаксиса, позволяя писать код, который на этапе компиляции превращается в другие Lisp-выражения. Это требует специальных фаз обработки кода, что делает компиляцию более сложной, но и более мощной.

4. Динамическая типизация и гибкая модель выполнения. Это усложняет статическую оптимизацию, но современные компиляторы используют техники сборки и статического анализа для улучшения производительности.

Современные тенденции и инструменты

1. Гибридные подходы. Большинство современных языков (в том числе Lisp-диалекты) используют комбинированный подход: исходный код может интерпретироваться, компилироваться в байт-код, а при необходимости – в нативный машинный код.

2. JIT-компиляция (Just-In-Time) – техника, при которой часть кода компилируется во время выполнения, обеспечивает баланс между быстрым запуском и высокой производительностью. Например, SBCL использует JIT для ускорения выполнения Lisp-программ.

3. Многоуровневая оптимизация. Современные компиляторы используют множество стадий оптимизации, включая статический анализ, встраивание функций, устранение избыточных вычислений и другим методам оптимизации для повышения скорости работы с функциями и представлениями данных.

1.5 История развития методов оптимизации компиляторов

Первые компиляторы появились в 1950-х годах. Они фокусировались на корректном переводе кода, а не на его оптимизации. В 1960-х годах разработчики осознали необходимость оптимизаций. Компилятор FORTRAN от IBM в конце 1960-х ввёл базовые техники, такие как удаление мертвого ко-

да. Книга «The Design of an Optimizing Compiler» 1975 года описала BLISS-компилятор, который применял глобальные оптимизации.

В 1970-х годах оптимизации стали стандартом. Компиляторы для языков вроде C и Pascal включали локальные преобразования. Развитие RISC-процессоров в 1980-х годах стимулировало внедрение оптимизаций. Процессоры с конвейерной обработкой требовали код, который учитывает задержки инструкций. Компиляторы адаптировались к таким особенностям.

В 1990-х годах появились межпроцедурные оптимизации. GCC и LLVM ввели уровни оптимизации, обозначенные флагами вида -O3, активирующие наборы техник. Профильно-ориентированная оптимизация (PGO) использует данные о выполнении программы для улучшения кода. Исследования также фокусировались на параллелизме для многоядерных систем.

В 2000-х годах машинное обучение стало использоваться для оптимизации. LLVM стал платформой для экспериментов с новыми методами. Развитие мобильных устройств подчеркнуло оптимизации для энергосбережения. Техники вроде векторизации эксплуатируют SIMD-инструкции.

Современная история включает интеграцию с ИИ. Компиляторы анализируют код с помощью нейронных сетей для выбора оптимизаций. Проекты вроде TensorFlow Compiler используют специализированные методы для машинного обучения. Эволюция отражает переход от ручных настроек к автоматизированным системам.

1.6 Основные концепции оптимизации компиляторов

Оптимизация компиляторов основана на преобразованиях, которые сохраняют семантику программы. Преобразования применяются к промежуточному представлению (IR), такому как трёхадресный код или SSA-форма. SSA (Static Single Assignment) присваивает каждой переменной значение только один раз, что упрощает анализ.

Ключевой концепцией служит анализ потока данных. Он определяет, как значения распространяются через программу. Анализ включает живые

переменные, достижимые определения и константные значения. Эти данные позволяют устранять избыточные вычисления.

Другая концепция – анализ контрольного потока. Граф контрольного потока (CFG) моделирует ветвления и циклы. Оптимизации используют CFG для выявления недостижимого кода или слияния блоков.

Профильно-ориентированная оптимизация собирает статистику выполнения. Инструменты вроде `perf` или `gprof` предоставляют данные о горячих точках. Компилятор перестраивает код на основе этих данных, перемещая редкие ветви в конец.

Концепция уровней оптимизации определяет наборы техник. Уровень -O0 отключает оптимизации для отладки. Уровень -O3 активирует агрессивные преобразования, включая векторизацию и встраивание функций.

Оптимизации учитывают компромиссы. Улучшение скорости может увеличить размер кода. Оптимизации для размера (-Os) минимизируют бинарный файл. Концепция безопасности гарантирует, что преобразования не вводят ошибки.

В функциональных языках оптимизации включают хвостовую рекурсию. Это преобразует рекурсию в цикл для экономии стека.

1.7 Классификация методов оптимизации

Методы оптимизации классифицируют по нескольким критериям:

1. В зависимости от машины. Независимые от машины методы работают с промежуточным кодом без учета архитектуры. Они включают константное сворачивание и удаление мертвого кода. Зависимые от машины методы адаптируют код к процессору, включая выбор инструкций и планирование.

2. По области применения. Локальные оптимизации ограничиваются базовым блоком – последовательностью инструкций без ветвлений. Они требуют минимального анализа. Глобальные оптимизации охватывают функцию, анализируя поток данных через блоки. Межпроцедурные оптимизации рассматривают всю программу, включая инлайнинг функций.

3. По типу преобразования. Оптимизации циклов фокусируются на петлях, которые занимают основное время выполнения. Техники включают развертку циклов и перемещение инвариантов. Оптимизации на основе потока данных устраняют избыточные выражения. SSA-оптимизации используют форму SSA для глобальной нумерации значений.

4. По фазе компиляции. Оптимизации фронтенда обрабатывают исходный код, включая макросы. Бэкенд-оптимизации генерируют машинный код, включая аллокацию регистров.

1.8 Обзор основных методов оптимизации

1. Константное сворачивание. Вычисляет выражения с константами на этапе компиляции. Пример: выражение $2 + 3$ заменяется на 5. Это сокращает инструкции и уменьшает время выполнения.

2. Распространение констант. Заменяет переменные их значениями при вычислении выражений, в которых используются эти переменные. Если переменная a равна 10, то $a * 2$ становится 20. Техника сочетается со сворачиванием.

3. Удаление мертвого кода. Устраняет инструкции без влияния на результат. Анализ живых переменных выявляет такие инструкции. Пример: присваивание переменной, которую не используют.

4. Удаление недостижимого кода. Удаляет блоки без путей к ним. С помощью распространения констант можно избавиться от ветвления с константным условием.

5. Встраивание функций. Заменяет вызов функции её телом. Это экономит время на вызов, но увеличивает размер. Компиляторы ограничиваются встраиванием малых функций.

6. Развертка циклов. Копирует тело цикла несколько раз, увеличивая тем самым объём кода, но сокращая число проверок. Данная техника способствует конвейеризации в силу меньшего ветвления.

7. Перемещение инвариантов. Выносит за пределы цикла выражения, не зависящие от переменной цикла и значение которого не меняется в течение выполнения всего цикла.

8. Слияние циклов. Комбинирует циклы с одинаковым количеством итераций и не зависящие друг от друга в один общий цикл. Это позволяет эффективнее использовать кэш.

9. Векторизация. Преобразует скалярные операции в векторные. Для массивов суммирование использует SIMD-инструкции.

10. Планирование инструкций. Переставляет команды для минимизации задержек. Зависимые инструкции разделяются независимыми.

11. Аллокация регистров. Распределяет переменные по регистрам. Алгоритм графового раскрашивания моделирует конфликты.

12. Глобальная нумерация. Значений выявляет эквивалентные выражения. SSA-форма их упрощает.

13. Хвостовая оптимизация рекурсии. Превращает рекурсию в цикл. В функциональных языках это предотвращает переполнение стека.

14. Оптимизации для кэша. Улучшают локальность. Перестановка циклов меняет порядок доступа к массиву.

15. Профильно-ориентированные техники. Используют данные выполнения. Компилятор переставляет ветви по вероятности.

1.9 Оптимизации компиляции функциональных языков и Lisp

Основные методы оптимизаций функциональных языков программирования:

1. Оптимизация хвостовой рекурсии позволяет выполнять хвостовые вызовы без расширения стека вызовов и тем самым предотвращает переполнение стека при глубокой рекурсии. При компиляции хвостовой рекурсии вместо создания нового кадра стека происходит замена текущего кадра кадром следующего вызова, что превращает рекурсию в итерацию и тем самым экономит память.

2. Встраивание функций (inlining) – это замена вызова функции её телом. Имеет различное влияние на производительность. Если функция вызывается один раз, то встраивание функции является очевидным решением, поскольку оно устраняет дополнительный вызов и упрощает выполнение кода. Если же функция вызывается много раз, то компилятор должен оценить, стоит ли делать подстановку функции. Это зависит от того, насколько эта подстановка повлияет на размер кода и скорость выполнения программы.

3. Structural Sharing – это техника оптимизации неизменяемых структур данных, заключающаяся в том, что при создании новой версии структуры вместо полного копирования старой версии происходит её повторное использование. Например, после добавления нового элемента в список этот элемент ссылается на старую часть списка. Это позволяет эффективнее использовать память, а также существенно улучшает скорость работы программ за счёт сокращения времени копирования структур.

4. Часто функциональные языки реализуют стратегию ленивых, или отложенных вычислений, согласно которой следует откладывать вычисления выражений до момента обращения к их значениям. Эта стратегия позволяет избежать ненужных вычислений, уменьшая нагрузку на процессор, и особенно полезна при работе с большими или бесконечными структурами данных. Однако ленивые вычисления не следует применять в языках, в которых присутствуют побочные эффекты, так как это может усложнить контроль над программой и привести к ошибкам. В чистых функциональных языках программирования отсутствуют побочные эффекты, поэтому для них использование отложенных вычислений является более естественным и предоставляет возможность писать чистый код.

5. Оптимизация копирования при записи (Copy-on-Write) является частным случаем ленивых вычислений и позволяет отложить копирование данных до момента их изменения. При дублировании структуры данных сначала передаётся ссылка на эти данные, вследствие чего данные становятся общими и разделяются между несколькими экземплярами. При попытке изменения одного из них происходит полное копирование исходных данных,

после чего экземпляр получает собственную независимую копию данных. Данная техника оптимизирует память, производя копирование только тогда, когда оно становится необходимым.

6. Метод удаления мёртвого кода представляет собой удаление участков кода, которые никогда не выполняются или не влияют на результат выполнения программы. Примером такого кода может служить функция, которая никогда не вызывается, ветка условия, которая никогда не вычисляется (при этом у компилятора есть достаточно информации, чтобы сделать такой вывод) или вычисление значения, которое нигде не используется. Данный метод уменьшает объём кода и время выполнения программы.

7. Техника удаления промежуточных структур снижает расход памяти путём объединения последовательных операций над структурами данных в общую операцию для избежания создания избыточных данных и их передачи между функциями. Эта оптимизация особенно актуальна в совокупности с ленивыми вычислениями, поскольку она компенсирует накладные расходы отложенных вычислений.

8. Удаление общих подвыражений заключается в обнаружении идентичных выражений для того, чтобы вычислить их один раз и затем повторно использовать полученный результат. Данный метод требует тщательный анализ программы на языке с побочными эффектами для точного выявления идентичных подвыражений. В силу отсутствия побочных эффектов в чистых функциональных языках программирования применение данной оптимизации особенно эффективно.

9. В качестве промежуточного представления кода при компиляции функциональных языков программирования зачастую используется стиль передачи продолжений (Continuation Passing Style), представляющий собой такой стиль написания кода, при котором каждая функция принимает дополнительный аргумент – продолжение, то есть функцию, которая указывает, что выполнять дальше с результатом текущей функции. Вместо возврата значения напрямую функция вызывает это продолжение, передавая ему вычисленный результат. Преобразование в стиль передачи продолжений происхо-

дит рекурсивно по всей программе, вследствие чего управляющий поток программы становится явным, а контроль над порядком вычислений и возвращением значений сосредоточен в функциях-продолжениях.

Особое место среди оптимизаций в Lisp занимает использование макросов, которые выступают важным инструментом для улучшения производительности и гибкости кода. Макросы разворачиваются уже на этапе компиляции, позволяя трансформировать и генерировать более эффективные программы. Это даёт возможность устранять избыточные вычисления и оптимизировать структуру кода ещё до его выполнения, обеспечивая тем самым уровень метапрограммирования, который является одной из ключевых особенностей Lisp и недоступен во многих других языках.

Несмотря на динамическую природу Lisp, современные компиляторы проводят довольно глубокий статический анализ. Например, в них активно используется добавление подсказок типов (type hints), что позволяет генерировать нативный код с меньшим количеством проверок во время выполнения. Такая стратегия уменьшает накладные расходы и ускоряет исполнение программ, сохраняя при этом гибкость динамической типизации.

Кроме того, важным аспектом оптимизации Lisp является эффективное управление памятью. Lisp-системы обычно применяют сборку мусора с поколенческим управлением, что существенно снижает задержки и повышает общую производительность системы. Компилятор, взаимодействующий со средой выполнения, может генерировать код, учитывающий особенности работы сборщика мусора, тем самым минимизируя накладные расходы, связанные с выделением и освобождением памяти.

2 Техническое задание

2.1 Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу бакалавра «Разработка программной системы компилятора функционального языка программирования Common Lisp с оптимизациями».

2.2 Цель и назначение разработки

Основной задачей выпускной квалификационной работы является разработка компилятора Common Lisp с оптимизациями и его тестирование.

Целью разработки является создание оптимизирующего компилятора Common Lisp.

Задачами данной разработки являются:

- изучение процесса компиляции языков программирования и функциональных языков в частности;
- изучение методов оптимизации компиляторов;
- проектирование компилятора с оптимизациями;
- реализация компиляции S-выражения в промежуточную форму;
- реализация генерации кода ассемблера;
- реализация генерации байт-кода;
- реализация оптимизации промежуточной формы.

2.3 Реализуемое подмножество Common Lisp

2.3.1 S-выражения

Компилируемое выражение представляет собой S-выражение – атом (символ, число, строка и др.) или список из нуля или более S-выражений.

2.3.2 Типы объектов

К объектам относятся числа: 5, 0xf, 3.14, символы: АВ, списки: (1 2 3), пары: (1 . 2), строки: "123", массивы: #(1 2 3), одиночные символы: #А, функции: #'(lambda (x) x). При переводе в логический тип NIL считается как ложь, всё остальное – истина (при выводе истины используется символ Т).

2.3.3 Специальные формы

1. (quote x) – особый оператор, который возвращает свой аргумент, не вычисляя его. Сокращенная запись – 'x (кавычка x).

2. (if <условие> <ветка true> <ветка false>). Условный оператор, считающий в своём условии истиной всё, кроме nil.

3. (setq <переменная₁> <выражение₁> ... <переменная_n> <выражение_n>). Создает глобальные переменные и присваивает им значения выражений. Возвращает значение последнего выражения.

4. (progn e₁ e₂ ... e_n). Последовательно вычисляет выражения e₁ ... e_n. Возвращает значение последнего выражения e_n.

5. (tagbody <список символов или выражений>). Последовательно вычисляет вложенные выражения. Символы выступают в роли меток, на которые можно перейти с помощью оператора go. Возвращает nil.

6. (catch <имя> <список выражений>). Блок с возможностью нелокального выхода. Сначала вычисляется имя блока, затем последовательно вычисляются выражения. Если встречается (throw <имя блока> <результат вычисления блока>), то вычисляется имя блока и происходит выход из ближайшего блока с этим именем. Если выход не происходит, то возвращается результат последнего выражения. Все имена вычисляются динамически.

7. (labels (<список функций>) <список выражений>). Объявляет локальные функции (в том числе и рекурсивные), после чего последовательно вычисляет выражения.

2.3.4 Прimitives

2.3.4.1 Списки

- (car x) – возвращает первый элемент списка (левый объект пары);
- (cdr x) – возвращает все кроме первого элемента списка (правый объект пары);
- (cons x y) – создает точечную пару с объектами x и y;
- (rplaca x y) – заменяет CAR списка x на значение y;
- (rplacd x y) – заменяет CDR списка x на значение y;
- (list $e_1 e_2 \dots e_n$) – создаёт список со значениями выражений $e_1 e_2 \dots e_n$.

2.3.4.2 Арифметические операции

- (+ x y) – сложение x и y;
- (- x y) – вычитание x и y;
- (* x y) – умножение x и y;
- (/ x y) – деление x и y;
- (sin num) – синус вещественного числа num;
- (cos num) – косинус вещественного числа num.

2.3.4.3 Предикаты

- (atom x) – возвращает Т (истина), если аргумент является атомом, иначе возвращает NIL, эквивалентный пустому списку (), что означает ложь;
- (eq x y) – возвращает Т, если x и y – один и тот же объект, иначе возвращает NIL;
- (and <список условий>) – логическая операция И;
- (or <список условий>) – логическая операция ИЛИ;
- (equal x y) – сравнение x и y;
- (> x y) – x больше чем y;
- (< x y) – x меньше чем y;
- (% x y) – остаток деления x и y;
- (pairp x) – предикат, является ли x парой;

- (symbolp x) – предикат, является ли x символом;
- (integerp x) – предикат, является ли x числом;
- (stringp x) – предикат, является ли x строкой;
- (arrayp x) – предикат, является ли x массивом;
- (floatp x) – предикат, является ли x числом с плавающей запятой.

2.3.4.4 Побитовые операции

- (& x y) – побитовое И от x и y;
- (bitor x y) – побитовое ИЛИ от x и y;
- (<< x y) – сдвиг влево x на y;
- (>> x y) – сдвиг вправо x на y.

2.3.4.5 Строки и символы

- (intern str) – создаёт символ на основе строки str;
- (symbol-name sym) – создаёт строку на основе символа sym;
- (string-size str) – возвращает длину строки str;
- (concat $s_1 \dots s_n$) – объединяет строки в одну общую строку (возвращает пустую строку при отсутствии аргументов);
- (inttostr x) – перевод целочисленного числа x в строку;
- (code-char x) – создает символ по коду x;
- (char-code c) – возвращает код символа c;
- (make-string len c) – создаёт строку длиной len, заполненную символом c;
- (char str i) – получает символ строки str по индексу i;
- (sets str i c) – заменяет символ в строке str с индексом i символом c;
- (subseq str a b) – получение подстроки из строки str (от индекса a до b).

2.3.4.6 Массивы

- (make-array x) – создание пустого массива длины x;
- (array-size x) – возвращает размер массива x;
- (aref x i) – чтение элемента массива x по индексу i;

- (set! x i y) – присвоение значения y элементу i массива x.

2.3.4.7 Печать объектов

- (print x) – печать объекта x;
- (putchar c) – печать символа c.

2.3.4.8 Другие примитивы

- (funcall <функция> <аргументы>) – вызывает функцию с аргументами;
- (apply <функция> <список аргументов>) – вызывает функцию с аргументами, переданными через список;
- (eval <выражение>) – вычисляет значение выражения как ещё одно выражение;
- (error msg) – останавливает вычисление, выводит сообщение об ошибке и возвращается в REPL цикл.

2.3.5 Лямбда выражения

Лямбда выражение – это анонимная функция #'(lambda (p₁ ... p_n) e₁ ... e_n), где p₁ ... p_n – это параметры функции, а e₁ ... e_n – выражения тела функции.

Вызов функции – это следующее выражение:

```
1 (funcall #'(lambda (p1 ... pn) e1 ... en) a1 ... an)
```

Сначала вычисляются все аргументы a₁ ... a_n. Затем каждому параметру p₁ ... p_n ставится в соответствие вычисленное значение аргументов a₁ ... a_n. После этого последовательно вычисляются выражения e₁ ... e_n, содержащие параметры, вместо которых будут подставлены их значения.

2.3.6 Определение функций

Новую функцию можно создать с помощью оператора defun:

```
1 (defun null (x)
2   (eq x ()))
```


2.3.7 Глобальные переменные

Глобальные переменные существуют все время работы. Они создаются с помощью функции (defvar имя_переменной [значение]). Значение может быть выражением:

```
1 (defvar a 10) -> A
2 A -> 10
```

При отсутствии значения в переменную записывается значение NIL.

Установить значение переменной можно с помощью функции setq (особая форма). Если такой переменной не было то она создается:

```
1 (setq x 1) -> 1
2 x -> 1
```

Можно одной функцией установить значения нескольких переменных:

```
1 (setq a 1 b 2 c 3)
```

Если переменная локальная (параметр функции), то setq ее модифицирует:

```
1 (defun test(x)
2   (setq x 10)) ; модификация параметра
```

Глобальные переменные являются динамическими, то есть их значение используется всегда последнее, которое было связано.

2.3.8 Макросы

Макрос задает шаблон для генерации выражения.

```
1 (defmacro test (var val)
2   (list 'setq var val))
```

При вызове макроса сначала происходит вычисление тела макроса (развертывание макроса):

```
1 (test abc 100) -> (setq abc 100)
```

Затем получившееся выражение вычисляется:

```
1 (setq abc 100) ; abc = 100
```

Обратная кавычка вычисляется как обычное цитирование, но также позволяет указывать, какие части цитирования должны быть вычислены. Эти части указываются с помощью запятой:

```
1 (setq a 10)
2 `(a b c ,a) -> (A B C 10)
```

Запятая-at (,@) служит для того, чтобы подставить список (результат вычисления выражения внутри запятой-at должен быть списком):

```
1 (defvar a '(1 2 3)) -> (1 2 3)
2 `(:,a ,@a) -> ((1 2 3) 1 2 3)
```

Можно посмотреть результат макроподстановки с помощью функции `macroexpand`:

```
1 (macroexpand '(if (= 1 1) 2 3)) -> (COND ((= 1 1) 2) (T 3))
```

2.3.9 Функции как объекты первого класса

Функции можно передавать в качестве параметров других функций и возвращать функции. Типы функций:

- локальные;
- глобальные;
- встроенные;
- лямбда.

Пространство имен функций не совпадает с пространством имен переменных. Функция передается в качестве параметра как лямбда-выражение или символ. Но при вызове функции из параметра мы должны указать, что

имя символа (переменная) относится к пространству имен функций. Для вызова функций по параметру используется примитив `funcall`. Пример:

```
1 ; вызов примитива
2 (funcall #' + 1 2 3) -> 6
3
4 ; вызов lambda
5 (funcall #'(lambda (x y) (+ x y)) 2 3) -> 5
6
7 ; вызов пользовательской функции по символу
8 (defun test (x) (+ x 1))
9 (funcall #'test 4) -> 5
```

Оператор `#'` является сокращением специальной формы `function`, создающей объект-замыкание, в котором сохраняется текущее окружение. При применении функции-замыкания будет использоваться не текущее окружение, а окружение, в котором было создано замыкание. Свободные локальные переменные являются лексическими, то есть сохраняют свое значение, которое было на момент замыкания. Глобальные переменные ведут себя динамическим образом – берется их значение в момент вызова функции.

2.3.10 Обработка ошибок

Обработка исключения для выражения:

```
1 (handle <выражение>
2   (error (<параметры исключения>)
3     <тело обработчика любых исключений в том числе просто error>)
4   (division-by-zero (c)
5     <тело обработчика деления на ноль>)
6   ...
7   (<пользовательское исключение> (<параметры>)
8     <обработчик пользовательского исключения>))
```

Для `error` существует обработчик по умолчанию, который выводит сообщение об ошибке. Примеры ошибок: `division-by-zero` (деление на ноль), `index-error` (выход за границы массива). Исключение может быть внутреннее или пользовательское:

```
1 (handle (if nil (raise 'my-error "My error" 20 30))
2   (my-error (params)
3   (print params)))
```

2.4 Программный интерфейс оптимизирующего компилятора

Программный интерфейс компилятора должен включать ряд функций, соответствующих каждому этапу компиляции:

1. (compile expr) – принимает на вход S-выражение expr и возвращает его промежуточное представление в виде дерева. Также функция собирает информацию о глобальных переменных и функциях для оптимизатора.

2. (optimize-tree tree) – принимает на вход промежуточное представление tree из предыдущего этапа, применяет оптимизации, указанные в глобальной переменной *optimize-flags* и возвращает оптимизированное дерево. Если убрать из глобальной переменной те или иные флаги, то выключатся соответствующие оптимизации. По умолчанию включены все поддерживаемые оптимизации.

3. (generate expr) – принимает на вход промежуточное представление expr из предыдущих этапов и генерирует на его основе список линейных инструкций ассемблера.

4. (assemble program) – принимает на вход линейный код ассемблера и трансформирует его в байт-код для выполнения на виртуальной машине в виде массива чисел.

После успешного выполнения каждой из выше перечисленных функций в указанном порядке на выходе имеется байт-код в совокупности со списком констант в глобальной переменной *consts*, размером памяти для глобальных переменных в *global-variables-count* и таблицей меток и соответствующих адресов в байт-коде в (cdr jmp-addr). В случае ошибки каждая функция может вызвать исключение comp-err вместе с сообщением об ошибке.

2.4.1 Флаги оптимизации компилятора

Функция compile должна принимать следующие флаги оптимизации:

2.4.1.1 trivial-condition

trivial-condition — оптимизация тривиальных условий. Пример:

```
1 (if t 1 2) -> 1
```

2.4.1.2 simplify-arithmetic

simplify-arithmetic — оптимизация арифметических выражений (замена примитивов с произвольным числом аргументов на примитивы с двумя аргументами). Пример:

```
1 (+ 1 2 3 4) -> (+ (+ (+ 1 2) 3) 4)
```

2.4.1.3 dead-code-elimination

dead-code-elimination — флаг, включающий сразу несколько оптимизаций.

К ним относятся:

1. Оптимизация загрузки регистра аккумулятора.

Удаление подряд идущих обращений к константам или переменным, которые не влияют на результат вычисления выражения. Пример:

```
1 (progn
2   1
3   2
4   (setq a 5)
5   a
6   3
7   4)
8 ->
9 (progn
10  (setq a 5)
11  4)
```

2. Удаление функций после подстановки.

Удаление объявлений функций, которые тело которых можно подставить во все места их вызова и для которых не создаётся замыкание. Пример:

```
1 (progn
2   (defun f (x) x)
3   (print (f 5)))
4 ->
5 (progn
6   (print 5))
```

2.4.1.4 tail-call

tail-call — оптимизация хвостовой рекурсии. Заменяет хвостовой рекурсивный вызов на безусловный переход, благодаря чему функция из рекурсивной превращается в итеративную. Например, следующая рекурсивная функция факториала:

```
1 (defun fact (n acc)
2   (if (= n 1)
3       acc
4       (fact (-- n) (* n acc))))
```

использует хвостовую рекурсию, так что при включенной оптимизации данная функция не будет заполнять стек при каждом рекурсивном вызове.

2.4.1.5 constant-folding

constant-folding — свёртка констант. Рекурсивно вычисляет значения примитивов, не имеющих побочных эффектов, с константными аргументами. Например:

```
1 (print (+ (5 * 2) 1)) -> (print 11)
```

2.4.1.6 unused-functions

unused-functions — удаление неиспользуемых функций. Удаляет объявления функций, которые ни разу не используются. Пример:

```

1 (progn
2   (defun f1 () 1)
3   (defun f2 () 2)
4   (defun f3 () 3)
5   (print (f3)))
6 ->
7 (progn
8   (defun f3 () 3)
9   (print (f3)))

```

2.4.1.7 beta-expansion

beta-expansion — встраивание функций. Подставляет тело функции вместо её вызова в случае, если эта функция вызывается единожды и удовлетворяет ряду требований, гарантирующих, что подстановка не изменит логику программы. Пример:

```

1 (progn
2   (defun f (x) x)
3   (print (f 5)))
4 ->
5 (progn
6   (defun f (x) x)
7   (print 5))

```

2.4.1.8 all-inline

all-inline — полное встраивание функций. Более агрессивная версия встраивания функций. Подставляет тело функции вместо её вызова независимо от числа вызовов данной функции в случае, если функция удовлетворяет выше упомянутому ряду требований возможности подстановки. Пример:

```

1 (progn
2   (defun f (x) x)
3   (print (f 5))
4   (print (f 10))
5   (print (f 15)))
6 ->
7 (progn
8   (defun f (x) x)
9   (print 5)
10  (print 10)
11  (print 15))

```

2.5 Требования к оформлению документации

Разработка программной документации и программного изделия должна производиться согласно ГОСТ 19.102-77 и ГОСТ 34.601-90. Единая система программной документации.

3 Технический проект

3.1 Общая характеристика организации решения задачи

Необходимо спроектировать и разработать оптимизирующий компилятор для языка Common Lisp, который способен транслировать исходное S-выражение в байт-код для целевой виртуальной машины. Компилятор должен поддерживать основные конструкции, а также применять ряд оптимизаций для повышения эффективности генерируемого кода.

Компилятор представляет собой программную систему, которая принимает на вход S-выражение и флаги оптимизации, преобразует выражение в промежуточное представление, применяет оптимизации и генерирует байт-код.

3.2 Архитектура оптимизирующего компилятора Common Lisp

Компилятор должен быть представлен в виде набора модулей, каждый из которых отвечает за ту или иную фазу компиляции. Должны быть определены следующие модули:

1. `common`. Включает в себя различные вспомогательные функции, которые используются другими модулями, но при этом тематически не подходят ни к одному из них.

2. `macro`. Включает в себя две основные функции – `(make-macro list)`, которая регистрирует новый макрос, используя переданные через список `list` название, параметры и тело макроса, и `(macroexpand args vals body)`, которая раскрывает макрос с телом `body`, заменяя символы `args` значениями `vals`, а также другие вспомогательные функции. Функция `macroexpand` возвращает S-выражение как результат раскрытого макроса, либо вызывает исключение в случае ошибки.

3. `compiler`. Состоит из функции `(compile expr)`, которая принимает на вход исходное S-выражение `expr` и производит его статический анализ, а также внутренние вспомогательных функций для анализа отдельных подвыражений. Встречая определение макроса или его раскрытие, `compile` вызывает

функции из модуля `macro`. Функция `compile` возвращает упрощённую промежуточную форму искомого S-выражения в случае успеха и вызывает исключение в случае ошибки.

4. `optimizer`. Состоит из функции (`optimize-tree tree flags`), которая принимает на вход промежуточную форму S-выражения `tree`, полученную на выходе из функции `compile`, и преобразует данную форму, применяя к ней оптимизации, указанные в качестве списка флагов оптимизаций `flags`, а также вспомогательных функций, отвечающих за конкретные оптимизации. Функция `optimize-tree` возвращает промежуточную форму с применёнными к ней оптимизациями, которая может остаться без изменений, если ни одна из указанных оптимизаций неприменима к данной промежуточной форме, либо если в качестве списка флагов оптимизаций на вход функции передан пустой список.

5. `generator`. Состоит из функции (`generate tree`), которая принимает на вход оптимизированную промежуточную форму S-выражения `tree`, полученную на выходе из функции `optimize`, и на её основе генерирует код ассемблера, состоящий из мнемоник инструкций, их операторов и меток для инструкций перехода, а также вспомогательных функций для генерации отдельных элементов промежуточной формы. Функция `generate` возвращает массив мнемоник инструкций, их операндов и меток.

6. `assembler`. Состоит из функции (`assemble code`), которая принимает на вход ассемблер код – массив мнемоник инструкций, операндов инструкций и меток для перехода, полученный на выходе из функции `generate`, и переводит его в байт-код для виртуальной машины, а также других вспомогательных функций. Функция `assemble` возвращает массив байтов, составляющих искомый байт-код, список констант, используемых в программе, размер памяти для глобальных переменных и таблицу названий функций и их адресов в случае успешного ассемблирования, а в случае ошибки – вызывает исключение.

3.3 Фаза 1. Макроподстановка

На данном этапе компиляции в исходном S-выражении регистрируются и раскрываются макросы. Макроподстановка представляет собой интерпретацию ограниченного подмножества Lisp, вычисляя в теле макроса всё, что можно во время компиляции. Это сохраняет выразительную силу макросов и в то же время является значительной оптимизацией компилятора.

Форма `defmacro` добавляет новый макрос с указанным именем, числом обязательных аргументов, списком параметров и телом в специальный список. Если макрос принимает фиксированное число аргументов, то он попадает в список `*fix-macros*`. Если макрос принимает переменное число аргументов, то он попадает в список `*nary-macros*`. В результате добавления нового макроса выражение `defmacro` исчезает из исходного S-выражения.

Вызов макроса состоит из поиска аргументов и тела соответствующего макроса, сопоставления фактических значений символам аргументов и последующего рекурсивного раскрытия тела макроса.

3.3.1 Атомарные выражения и окружение макросов

Константы (все атомарные значения, кроме символов) в процессе раскрытия остаются без изменений.

Для подстановки символов происходит симуляция окружения переменных, представляющее из себя список пар символов и значений. В начале раскрытия макроса окружение состоит из аргументов и их значений, оставленных в изначальном состоянии. Если какой-либо аргумент представлен в виде сложного выражения (специальная форма или применение примитива), то он остаётся в таком виде до тех пор, пока не понадобится его значение, в случае чего он будет вычислен.

В процессе раскрытия макроса окружение может быть расширено. В этом случае в начало списка окружения добавляются новые пары символов и их значений, а при восстановлении окружения — удаляются из начала.

Также должна быть представлена хеш-таблица **macro-globals**, в которой хранятся глобальные переменные, также представленные в виде пар символов и значений. Эти глобальные переменные существуют только на данном этапе компиляции и используются для сохранения значений между различными вызовами макросов.

При подстановке символа происходит поиск пары с соответствующим символом сначала в текущем окружении, а затем в таблице глобальных переменных, в следствие чего символ заменяется своим значением. Если такого символа не существует ни в окружении, ни среди глобальных переменных, то возвращается ошибка компиляции. Символы *T* и *NIL* являются особым случаем: значение символа *T* – тот же символ *T*, символа *NIL* – пустой список.

Примеры:

```
1 (progn
2   (defmacro m () 5)
3   (m))
4 ->
5 (progn
6   5)
7
8 (progn
9   (defmacro m () t)
10  (m))
11 ->
12 (progn
13   T)
14
15 (progn
16   (defmacro m (x) x)
17   (m 5))
18 ->
19 (progn
20   5)
```

3.3.2 Вызов примитива

Для вызова примитива вычисляются его аргументы, после чего происходит применение примитива к полученным значениями аргументов и возврат значения примитива. Если аргумент примитива – символ, происходит подстановка символа, чтобы получить значение аргумента. Если аргумент

сам является вызовом примитива, либо представлен в виде специальной формы, то происходит рекурсивное вычисление аргумента. Пример:

```
1 (progn
2   (defmacro m (x) (+ x 1))
3   (m 2))
4 ->
5 (progn
6   3)
```

3.3.3 Пользовательские функции

Определения функций вне макросов регистрируются в специальные списки в зависимости от переменности числа аргументов, аналогично макросам (*fix-functions* и *nary-functions*). Определения функций внутри макросов запрещены.

Если в процессе раскрытия макроса встречается вызов функции, определённой к этому моменту пользователем, то происходит вычисление аргументов, расширение окружения полученными значениями аргументов и раскрытие тела функции с расширенным окружением и с последующим возвратом вычисленного значения функции. Пример:

```
1 (progn
2   (defun list (&rest args) args)
3   (defmacro m (f) (list f 1 2 3))
4   (m '+))
5 ->
6 (progn
7   (defun list (&rest args) args)
8   (+ 1 2 3))
```

3.3.4 if

Для вычисления значения условной формы if происходит вычисление значения условия, на основе которого дальше вычисляется одна из веток — ветка «истина» и ветка «ложь». Значением формы является значение вычисленной ветки. Пример:

```

1 (progn
2   (defmacro m (x) (if x 1 2))
3   (m t)
4   (m nil))
5 ->
6 (progn
7   1
8   2)

```

3.3.5 setq

Форма `setq` позволяет создавать и модифицировать глобальные переменные, существующие на уровне макросов. Пример:

```

1 (progn
2   (defmacro init () (setq count 0))
3   (init)
4
5   (defmacro m () (setq count (+ count 1)))
6   (m)
7   (m)
8   (m))
9 ->
10 (progn
11   0
12
13   1
14   2
15   3)

```

3.3.6 progn

Форма `progn` поочерёдно вычисляет данную ей последовательность выражений и возвращает в качестве результата значение последнего выражения. Пример:

```

1 (progn
2   (defmacro m ()
3     (progn
4       (setq x 1)
5       (setq x (+ x 1))))
6   (m))
7 ->
8 (progn
9   2)

```

3.3.7 let

Форма `let` принимает на вход список пар символов и значений, вычисляет значения символов, расширяет окружение этими парами и поочерёдно выполняет внутренние выражения с расширенным окружением. По выходе из формы окружение восстанавливается. Пример:

```
1 (progn
2   (defmacro m (x)
3     (let ((y (* x 2)))
4       (+ y 1)))
5   (m 5))
6 ->
7 (progn
8   11)
```

3.3.8 cond

Форма `cond` принимает на вход список пар, левый элемент которых — условие, а правый — следствие. Форма последовательно вычисляет значение условия каждой пары по очереди, пока одно из этих условий не вернёт истинное значение, после чего вычисляется и возвращается значение следствия соответствующей пары. Если ни одно из условий не вернуло истину, результатом формы будет `nil`. Пример:

```
1 (progn
2   (defmacro sign (x)
3     (cond
4       ((< x 0) -1)
5       ((> x 0) 1)
6       (t 0)))
7   (sign 5))
8 ->
9 (progn
10  1)
```

3.3.9 quote

Форма `quote` возвращает свой аргумент, не вычисляя его. Пример:

```

1 (progn
2   (defmacro hello () '(print "hello"))
3   (hello))
4 ->
5 (progn
6   (print "hello"))

```

3.3.10 backquote

Форма `backquote` возвращает свой аргумент, если он атом или пустой список. Если аргумент содержит форму `comma`, то вычисляется выражение внутри `comma`, после чего его значение подставляется в результирующее выражение. Если аргумент – список, первый элемент которого – форма `comma-at`, то результат вычисления выражения внутри `comma-at` объединяется с оставшейся вычисленной частью списка. Если аргумент – произвольный список, то форма по очереди вычисляет каждый элемент списка. Пример:

```

1 (progn
2   (defun null (x) (eq x ()))
3   (defun list-length (list)
4     (if (null list)
5         0
6         (+ (list-length (cdr list)) 1)))
7   (defmacro mean (&rest xs)
8     `(/ (+ ,@xs) ,(list-length xs)))
9   (mean 1 2 3))
10 ->
11 (progn
12   (/ (+ 1 2 3) 3))

```

3.3.11 function

Форма `function` вычисляет и возвращает функциональный объект по символу или лямбда-выражению. Для символа производится поиск функции в списке функций. Для лямбда-выражения производится проверка на правильную структуру лямбда-выражения. Пример:


```

1 (progn
2   (defmacro add (&rest args) (apply #' + args))
3   (add 1 2 3))
4 ->
5 (progn
6   6)

```

3.3.12 funcall

Форма `funcall` выполняет данный ей объект замыкания лямбда-выражения, расширяя при этом окружение данными параметрами. Пример:

```

1 (progn
2   (defmacro m () (funcall #'(lambda () (+ 1 2 3))))
3   (m))
4 ->
5 (progn
6   6)

```

3.4 Фаза 2. Статический анализ

На данном этапе компиляции входное S-выражение преобразуется в другое абстрактное синтаксическое дерево, состоящее из простых операций и сохраняющее смысл исходного выражения.

3.4.1 Макросы

Форма `defmacro` обрабатывается модулем `macro`, а в искомое дерево вместо определения макроса попадает форма `NOP` – пустой оператор, который означает отсутствие действия и впоследствии будет удалён на следующих этапах компиляции.

При обработке вызова функции в случае, если функция – макрос, происходит раскрытие макроса в модуле `macro` и затем происходит анализ полученного выражения.

3.4.2 Лексическое окружение

Для статического разрешения переменных используется лексическое окружение. Оно представляет собой набор кадров, каждый из которых содер-

жит список связанных переменных. В начале компиляции окружение пустое. При анализе определения функции окружение расширяется новым кадром, состоящим из аргументов данной функции. По окончании обработки определения функции кадр этой функции удаляется из окружения. Таким образом, происходит симуляция поведения окружения при выполнении программы на этапе компиляции, чтобы заранее определить положения переменных в окружении и избавиться от поиска переменных по их имени во время выполнения программы.

3.4.3 Константы и переменные

Константы переносятся в искомое дерево без изменений в форму CONST. Пример:

```
(CONST 86)
```

При анализе переменной определяется её тип. Должно быть определено 3 типа переменных:

1. Глобальные переменные. Каждая глобальная переменная заносится в отдельный список **global-vars**. Обращение к таким переменным происходит по индексу переменной в этом списке с помощью формы GLOBAL-REF:

```
(GLOBAL-REF 0) ; получить значение глобальной переменной со смещением 0
```

В начале компиляции в список глобальных переменных добавляются символы T и NIL. С учётом этого, память для глобальных переменных виртуальной машины должна иметь размер как минимум в 2 элемента, первый из которых должен по умолчанию иметь значение символа T, второй – значение пустого списка. Таким образом, обращение к символу T принимает вид (GLOBAL-REF 0), а к символу NIL или пустому списку – (GLOBAL-REF 1).

2. Локальные переменные – переменные, определённые в текущем кадре активации. Эти переменные заносятся в текущий кадр лексического окружения, и обращение к ним происходит по смещению в этом кадре:

```
1 (LOCAL-REF 0) ; получить значение локальной переменной со смещением в кадре 0
```

3. Свободные переменные – переменные, определённые в одном из более верхних кадров активации по отношению к текущему. Эти переменные также заносятся в лексическое окружение, и обращение к ним происходит с помощью указания смещения относительно текущего кадра и смещения внутри кадра:

```
1 (DEEP-REF 1 0) ; получить значение свободной переменной со смещением кадра 1  
и со смещением в кадре 0
```

Таким образом, в результате статического анализа все обращения к переменным будут заменены на смещения, благодаря чему не придётся проводить их поиск во время выполнения программы.

3.4.4 progn

Последовательность выражений `progn` преобразуется в список, в котором первый элемент – символ `SEQ`, а остальные – операции промежуточной формы, соответствующие подвыражениям исходного `progn` в том же порядке появления. Для `progn` с единственным подвыражением вместо формы `SEQ` генерируется только операция для вычисления подвыражения. Пустой `progn` преобразуется в вычисление пустого списка. Пример:

```
1 (progn 1 2 3) -> (SEQ (CONST 1) (CONST 2) (CONST 3))  
2 (progn 1) -> (CONST 1)  
3 (progn) -> (CONST ())
```

3.4.5 if

Условная форма `if` трансформируется в список, первый элемент которого – символ `ALTER`, второй – операция для вычисления условия, третий – операция для вычисления по первой ветке, четвёртый – по второй ветке. Пример:

```
1 (if t 1 2) -> (ALTER (GLOBAL-REF 0) (CONST 1) (CONST 2))
```

3.4.6 `setq`

При анализе присваивания `setq` определяется тип переменной, затем генерируется соответствующая операция присваивания (`GLOBAL-SET` для глобальных переменных, `LOCAL-SET` — для локальных и `DEEP-SET` — для свободных) вместе с рекурсивной генерацией подвыражения. Пример:

```
(setq x 5) -> (GLOBAL-SET 2 (CONST 5))
```

3.4.7 Функции и замыкания

Пользовательские функции с фиксированным количеством аргументов обрабатываются аналогично лямбда-выражениям. Имя функции, смещение кадра окружения, количество аргументов сохраняется в таблицу `*fix-functions*` для последующей проверки при вызове. Для лямбда-функций имена генерируются автоматически.

Функции с переменным числом аргументов сохраняются в таблицу `*nary-functions*`, в которой хранится имя функции, смещение кадра окружения и число фиксированных аргументов (остальные аргументы добавляются в виде списка).

При обработке формы `labels` текущее лексическое окружение расширяется определениями локальных функций. Сохраняется имя самой локальной функции и автоматически сгенерированное имя функции. Далее генерируется код для этих функций, после чего окружение восстанавливается к предыдущему состоянию.

Примитивы с фиксированным числом аргументов хранятся в таблице `*fix-primitives*`. Хранится имя примитива и число аргументов.

Примитивы с переменным числом аргументов хранятся в таблице `*nary-primitives*`. Хранится имя примитива и число фиксированных аргументов.

При компиляции применения функции, определяется тип функции: глобальная, лямбда или примитив. Затем происходит проверка числа аргу-

ментов. Для лямбда-функций перед вызовом вставляется скомпилированное тело функции. Перед вызовом функции активируется замкнутое окружение функции, создаётся кадр активации с заранее вычисленным размером, вычисляются слева направо аргументы и заносятся в текущий кадр. При вызове примитивов кадр активации не создается.

Для компиляции тела функций используется форма FUNC, которая представляет собой именованную метку, на которую можно совершить переход, что можно использовать для выполнения функций в байт-коде. Для генерации определения функций можно использовать имя функции в качестве названия метки.

Для перехода на метку функции с фиксированным числом аргументов используется форма FIX-CALL, для функции с переменным числом аргументов — NARY-CALL. Форма FIX-CALL содержит название метки, номер кадра окружения, в котором была определена функция и относительно которой нужно создавать новый кадр, и список операций, каждая из которых вычисляет аргумент функции, в порядке передачи аргументов функции. В форме NARY-CALL дополнительно содержится количество фиксированных аргументов функции.

Для возврата из функции обратно в место после вызова функции используется форма RETURN.

Пример промежуточной формы определения функции через defun и её вызова:

```
1 (defun f (x)
2   x)
3 ->
4 (SEQ
5   (LABEL F
6     (SEQ
7       (LOCAL-REF 0)
8       (RETURN)))
9   (FIX-CALL F 0 ((CONST 5)))) ; переход на метку функции с фиксированным
                                ; числом аргументов, с переходом на кадр под номером 0 и со списком
                                ; аргументов, состоящих из константы 5
```

3.4.7.1 tagbody

Форма tagbody преобразуется в форму SEQ с заменой символов на формы LABEL и формы go на GOTO – безусловный переход на метку. Пример:

```
1 (tagbody
2   (go skip)
3   1
4   2
5   skip
6   3)
7 ->
8 (SEQ
9   (GOTO G1)
10  (CONST 1)
11  (CONST 2)
12  (LABEL G1
13    (CONST 3)))
```

3.4.8 catch/throw

Форма catch преобразуется в аналогичную форму CATCH, и таким же образом форма throw превращается в форму THROW. Пример:

```
1 (catch 'test
2   (throw 'test 5))
3 ->
4 (CATCH (CONST TEST) (THROW (CONST TEST) (CONST 5)))
```

3.4.9 Операции после первой фазы

В таблице 3.1 представлены все команды, входящие в промежуточное представление.

Таблица 3.1 – Операции после первой фазы

Операция	Описание операции
1	2
NOP	Нет операции.
CONST expr	Выражение константа.
GLOBAL-REF i	Обращение к i-той глобальной переменной.
LOCAL-REF j	Обращение к j-той локальной переменной.

Продолжение таблицы 3.1

1	2
DEEP-REF $i\ j$	Обращение к дальней свободной переменной. i – смещение кадра активации, j – позиция в кадре.
GLOBAL-SET $i\ \text{expr}$	Присваивание выражения в i -тую глобальную переменную.
LOCAL-SET $j\ \text{expr}$	Присваивание выражения в j -тую локальную переменную.
DEEP-SET $i\ j\ \text{expr}$	Присваивание выражения в дальнюю свободную переменную. i – смещение кадра активации, j – позиция в кадре.
SEQ $\langle \text{expr list} \rangle$	Последовательность вычислений списка выражений.
ALTER cond true false	Условный оператор с выражениями условия, истины и лжи.
FUNC name body	Функция с именем name и телом body .
LABEL name	Метка с именем name .
FIX-LET num args expr	let форма, num – число аргументов, args – список выражений аргументов, expr – тело λ .
FIX-CLOSURE name body	Замыкание функции с именем name и телом body .
PRIM-CLOSURE name	Замыкание примитива (фиксированное число аргументов) с именем name .
NPRIM-CLOSURE name	Замыкание примитива (переменное число аргументов) с именем name .
RETURN	Выход из функции.
FIX-CALL name env args	Вызов функции с именем name , номером окружения ofs и списком выражений аргументов args (фиксированное число аргументов).

Продолжение таблицы 3.1

1	2
NARY-CALL name num env args	Вызов функции с именем name, номером окружения ofs и списком выражений аргументов args (переменное число аргументов). num - число фиксированных аргументов.
TAIL-CALL name env args depth	Хвостовой вызов функции с именем name, номером окружения env, списком выражений аргументов args и смещением в окружении вызова функции от её определения depth (фиксированное число аргументов).
TAIL-NCALL name num env args depth	Хвостовой вызов функции с именем name, номером окружения env, списком выражений аргументов args и смещением в окружении вызова функции от её определения depth (переменное число аргументов). num – число фиксированных аргументов.
FIX-PRIM name args	Вызов примитива (фиксированное число аргументов) с именем name и списком выражений аргументов args.
NARY-PRIM name num args	Вызов примитива (переменное число аргументов) с именем name, числом фиксированных аргументов num и списком выражений аргументов args.
NARY args	Генерация списка необязательных параметров args.
GOTO name	Переход на метку name.
CATCH tag_expr body_expr	Блок catch с выражением метки tag_expr и множеством выражений body_expr.
THROW tag_expr ret_expr	Выход из блока catch с выражением метки tag_expr и результатом вычислений ret_expr.

Продолжение таблицы 3.1

1	2
APPLY func args pack	Применение вычисляемой функции func к списку аргументов args, pack – нужно ли соединить аргументы в список.

3.5 Фаза 3. Оптимизация

На данном этапе компиляции промежуточное дерево, получившееся в результате статического анализа, преобразуется посредством применения указанных оптимизаций. Оптимизация выражения состоит из последовательного вызова функций, соответствующих включенным флагам оптимизации.

3.5.1 Оптимизация тривиальных условий (trivial-condition)

Форму ALTER, условие которой – константа, можно упростить, вычислив условие на этапе компиляции и заменив всю форму одной из её веток. Примеры:

```

1 (ALTER (CONST T)
2   (CONST 1)
3   (CONST 2))
4 ->
5 (CONST 1)
6
7 (ALTER (CONST NIL)
8   (CONST 1)
9   (CONST 2))
10 ->
11 (CONST 2)

```

3.5.2 Оптимизация арифметических выражений (simplify-arithmetic)

Заменяет примитивы с произвольным числом аргументов на примитивы с двумя аргументами. Пример:

```

1 (+ 1 2 3 4)
2 ->
3 (+ (+ (+ 1 2) 3) 4)

```

3.5.3 Удаление неиспользуемого кода (dead-code-elimination)

Включает в себя несколько оптимизаций.

3.5.3.1 Оптимизация загрузки регистра аккумулятора

Последовательности идущих подряд обращений к константам или переменным, встречающиеся в формах SEQ, преобразуются таким образом, что все, кроме последнего обращения в последовательности, а также обращения перед командой RETURN, удаляются. К таким обращениям относятся CONST, GLOBAL-REF, LOCAL-REF и DEEP-REF. Пример:

```

1 (SEQ
2   (GLOBAL-SET 2 (CONST 5))
3   (CONST 10)
4   (GLOBAL-REF 2)
5   (GLOBAL-REF 1))
6 ->
7 (SEQ
8   (GLOBAL-SET 2 (CONST 5))
9   (GLOBAL-REF 1))

```

Данная оптимизация также удаляет операции обращения к переменным, если непосредственно перед такой операцией находится присваивание этой же переменной, так как результат вычисления операции присваивания равен значению, которое было присвоено переменной в ходе этой операции, поэтому последующее обращение к этой переменной избыточно. К операциям присваивания относятся GLOBAL-SET, LOCAL-SET и DEEP-SET. Пример:

```

1 (SEQ
2   (GLOBAL-SET 2 (CONST 5))
3   (GLOBAL-REF 2))
4 ->
5 (SEQ
6   (GLOBAL-SET 2 (CONST 5)))

```

3.5.3.2 Удаление функций после подстановки

В первой фазе собирается информация о возможности подстановки функций. Во второй фазе для функций, которые вызываются один раз и могут быть подставлены, удаляется узел LABEL. Но удаление невозможно, если:

- LABEL расположен внутри FIX-CLOSURE (замыкание lambda-функции);
- замыкание пользовательской функции - FIX-CLOSURE без LABEL (для этого необходимо собрать информацию о замыкании функции в первом проходе).

3.5.4 Оптимизация хвостовой рекурсии (tail-call)

Хвостовые вызовы TAIL-CALL и TAIL-NCALL не должны увеличивать число кадров активации, то есть они используют текущий кадр, изменяя параметры вызова в нем. При хвостовом вызове не нужно также сохранять и восстанавливать текущий кадр активации, поэтому хвостовой вызов транслируется в JMP.

Чтобы найти вызовы из хвостовых позиций в функцию inner-compile добавляется еще один логический аргумент tail, который означает, расположено ли текущее выражение в хвостовой позиции.

Основные правила определения хвостовой позиции:

1. Форма (setq x e). Выражение e – не хвостовой вызов, потому что оно записывается в переменную, то есть не является последним действием текущего блока.
2. Форма (if e t f). Выражение e – не хвостовой вызов, т.к. после его вычисления следует вычисление одной из веток условия.
3. Форма (progn $x_1 \dots x_n$). Все выражения, кроме последнего (x_n) – не хвостовые вызовы.
4. Применение функции – всегда хвостовой вызов для тела функции.

3.5.5 Свёртка констант (constant-folding)

Константные выражения считаются на этапе компиляции. Пример:

```
(setq x (+ 1 2 3)) -> (setq x 6)
```

Сворачиваются только примитивы, не имеющие побочных эффектов, кроме `make-array` и `make-string`.

3.5.6 Удаление неиспользуемых функций (**unused-functions**)

В первой фазе компиляции для всех функций (глобальных и локальных) подсчитывается число ссылок на них. Ссылка может быть как вызов функции (применение функции) или как создание замыкания (объекта-функции). Если у функции 0 ссылок, значит она не используется в коде, и ее можно удалить.

3.5.7 Встраивание функций, которые вызываются один раз (**beta-expansion**)

Происходит во второй фазе оптимизации, потому что тела функций могут быть переопределены.

Встраивание возможно, когда:

- функция не рекурсивная;
- нет установки локальных переменных (`LOCAL-SET`);
- к каждому аргументу идет обращение только один раз;
- в выражении аргументов нет побочных эффектов;
- в функции есть замыкание, в теле которого нет `DEEP-REF`;
- нет повторного определения функции.

Вместо вызова функции можно подставить скомпилированное тело функции. В теле функции необходимо заменить обращения к переменным:

- локальные переменные (`LOCAL-REF`) заменяются на скомпилированные выражения аргументов;
- свободные переменные понижаются в уровне обращения (`DEEP-REF/SET 2` заменяется на `DEEP-REF/SET 1`, `DEEP-REF/SET 1` заменяется на `LOCAL-REF/SET`) (не внутри `FIX-LET`);
- `RETURN` заменяется на `NOP`.

При подстановке внутри замыканий (FIX-CLOSURE) необходимо учитывать относительный уровень глубины вызова.

Для того, чтобы была возможна каскадная подстановка, новое оптимизированное тело сохраняется.

Лямбда функции FIX-LET всегда вызываются один раз, и их тоже можно встроить по тем же условиям.

Встраивание функций с переменным числом аргументов аналогично встраиванию обычных функций за исключением того, что необходимо собирать необязательные аргументы в список в месте подстановки.

3.5.8 Полное встраивание функций (all-inline)

Аналогично оптимизации beta-expansion за исключением того, что решается также встраивать те функции, которые вызываются более одного раза.

3.6 Фаза 4. Генерация линейных инструкций

Дерево, полученное на этапе компиляции, преобразуется в ассемблер код – список инструкций с мнемониками вместо опкодов. Для этого элементы дерева, начиная с самого верхнего, рекурсивно генерируются определённым образом.

Инструкции после генерации представлены в таблице 3.2:

Таблица 3.2 – Инструкции после генерации

Инструкция	Описание инструкции
1	2
CONST val	Записать val в аккумулятор.
GLOBAL-REF i	Записать i-тую глобальную переменную в аккумулятор.
LOCAL-REF j	Записать j-тую локальную переменную в аккумулятор.

Продолжение таблицы 3.2

1	2
DEEP-REF $i\ j$	Записать дальнюю свободную переменную в аккумулятор. i – смещение кадра активации, j – позиция в кадре.
GLOBAL-SET i	Записать значение аккумулятора в i -тую глобальную переменную.
LOCAL-SET j	Записать значение аккумулятора в j -тую локальную переменную.
DEEP-SET $i\ j$	Записать значение аккумулятора в дальнюю свободную переменную. i – смещение кадра активации, j – позиция в кадре.
PUSH	Помещаем аккумулятор в стек.
POP	Извлекаем значение из стека и записываем в аккумулятор.
PRIM prim	Вызова примитива с фиксированным числом аргументов с именем prim.
NPRIM nprim	Вызова примитива с переменным числом аргументов с именем nprim.
FUNC f	Начало тела функции f .
LABEL name	Символьная метка.
JMP label	Безусловный переход на метку label.
JMPTAIL label	Переход при хвостовом рекурсивном вызове.
JNT label	Переход на метку если в аккумуляторе nil.
ALLOC n	Создать новый кадр активации с размером n . Записать туда значения из стека в обратном порядке.
REG-CALL f	Вызов функции f .
SAVE-ENV	Сохранить текущий кадр активации в стеке.
SET-ENV num	Установить кадр активации с позицией num от начала списка кадров.
RESTORE-ENV	Восстановить кадр активации из стека.
FIX-CLOSURE name	Создать замыкание с текущим кадром активации, кодом по метке name, записать замыкание в аккумулятор.

Продолжение таблицы 3.2

1	2
PRIM-CLOSURE name	Создать замыкание с текущим кадром активации, именем примитива name (фиксированное число аргументов), записать замыкание в аккумулятор.
NPRIM-CLOSURE name	Создать замыкание с текущим кадром активации, именем примитива name (переменное число аргументов), записать замыкание в аккумулятор.
RETURN	Выход из функции.
CATCH ofs	Создать блок catch. Метка блока находится в аккумуляторе, конец блока по смещению ofs.
THROW	Нелокальный переход на конец блока catch с меткой на вершине стека и результатом вычисления в асс.
FUNC-CALL	Вызывает функцию-замыкание на вершине стека.
CHECK-PRIM	Является ли объект на вершине стека замыканием примитива.
PRIM-CALL	Вызывает функцию-замыкание как примитив на вершине стека.

Для элементов CONST, GLOBAL-REF, LOCAL-REF, DEEP-REF и RETURN существуют соответствующие инструкции, поэтому они генерируются как инструкции с мнемониками и операндами без изменений.

Для элементов GLOBAL-SET, LOCAL-SET и DEEP-SET сначала генерируется вычисление их аргументов, при этом идёт симуляция глобальных переменных и окружения, чтобы затем эти инструкции можно было сгенерировать с правильными аргументами.

Элемент FUNC используется для генерации тела функции, поэтому сначала рассчитывается метка после тела функции, генерируется переход на эту метку, после этого генерируется тело самой функции, и в конце добавляется эта метка.

В элементе SEQ для каждого дочернего элемента рекурсивно по очереди происходит дальнейшая генерация.

Для элемента ALTER рассчитываются метки для ветки по лжи и для конца if-блока, затем генерируется условие, условный переход на ветку по лжи, тело по истине и безусловный переход на конец блока, метка и тело по лжи, и метка конца блока.

В элементе FIX-PRIM для каждого аргумента генерируется вычисление этого аргумента и инструкция PUSH, затем генерируется вызов соответствующего примитива PRIM.

В элементе NARY-PRIM для каждого аргумента генерируется вычисление этого аргумента и инструкция PUSH, необязательные аргументы собираются в список с помощью команды PASC (удаляются из стека и добавляются как список), затем генерируется вызов соответствующего примитива NPRIM.

Для элемента REG-CALL генерируется вычисление каждого аргумента и добавление в стек, установление соответствующего окружения для текущей функции (SET-ENV), создание кадра активации и добавления в окружение аргументов функции из стека (ALLOC), вызов самой функции (REG-CALL) и в конце восстановление окружения (RESTORE-ENV).

Для элемента APPLY генерируется вычисление каждого аргумента и добавление в стек через PUSH, опциональное объединение этих аргументов в список с помощью PASC, проверка функционального объекта в аккумуляторе на примитив, условный переход на вызов функции, вызов примитива, безусловный переход на конец блока, метка для вызова функции, сохранение текущего кадра окружения в стек, вызов функции, восстановление окружения и метка конца блока.

3.7 Фаза 5. Ассемблер

Последним шагом компиляции является генерация байт-кода из ассемблер-кода. Ассемблирование происходит в 2 прохода.

При первом проходе мнемоники опкодов заменяются соответствующими байтами опкодов, при этом если встречается метка, то она не добавляется в результирующий байт-код, но запоминается в хеш-таблицу с адресом следующей инструкции, и если встречается инструкция перехода, то в отдель-

ный список добавляется текущий адрес с меткой, которую использует данная инструкция.

Второй проход идёт по списку адресов с метками и в байт-коде заменяются соответствующие метки на адреса меток из хеш-таблицы.

На выходе компилятора:

- список констант;
- число глобальных переменных;
- массив с байт-кодом.

Точку входа в программу можно посчитать (перейти по всем JMP, до первого оператора не JMP).

3.8 Архитектура целевой виртуальной машины

Машина включает в себя память программы (где хранится байт код программы), память констант, память глобальных переменных, стек, список кадров активации и регистры.

На рисунке 3.1 представлена схема целевой виртуальной машины.

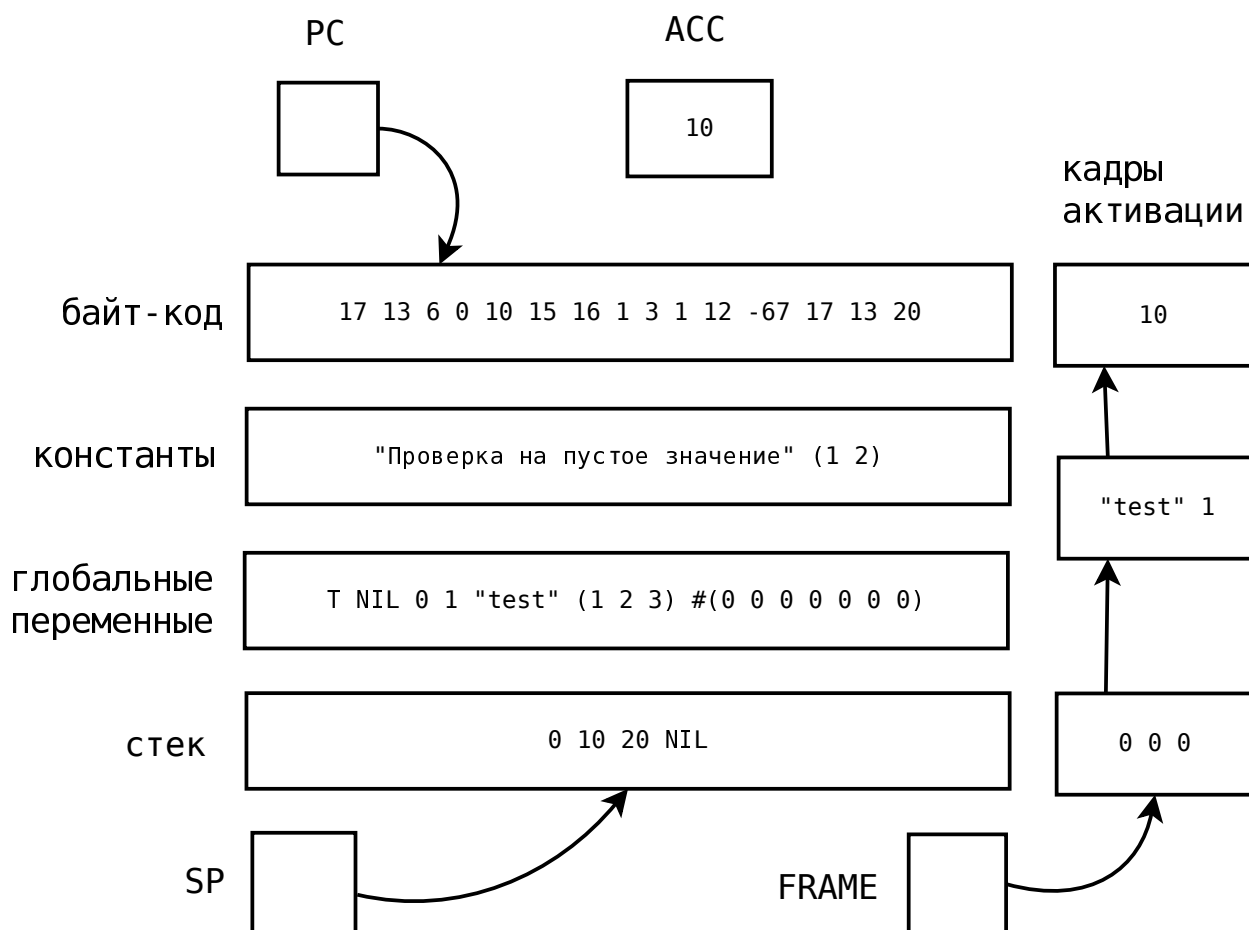


Рисунок 3.1 – Схема целевой виртуальной машины

В памяти программы хранятся инструкции программы в виде объектов NUMBER. Каждая команда состоит из кода операции и возможных параметров.

В памяти констант и глобальных переменных могут храниться объекты любых типов.

Стек может хранить объекты любых типов.

Кадры активации представляют собой объекты-массивы.

На рисунке 3.2 представлена схема кадров активации.

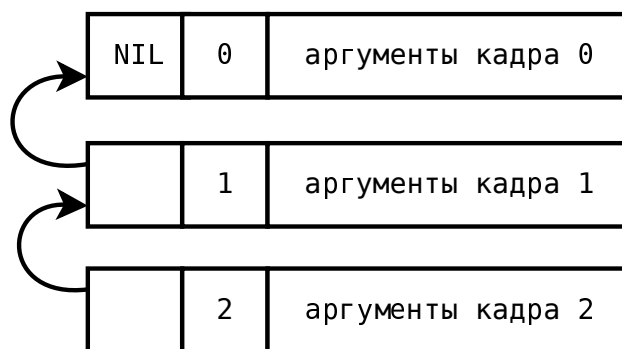


Рисунок 3.2 – Схема кадров активации

Первый элемент массива – это ссылка на предыдущий кадр. Следующий элемент служит для ускорения поиска кадра по глубине вызова, здесь хранится номер кадра (глубина вызовов). Остальные элементы массива – это локальные аргументы. Кадры могут иметь не только линейную, но и древовидную структуру, поэтому необходима ссылка на предыдущий кадр. Такие объекты удобно записывать в стек, восстанавливать из стека. Кадры активации создаются динамически как и другие объекты программы.

Для блоков `catch` существует отдельный стек, который хранит записи из имени метки, абсолютного адреса конца блока, номера кадра активации, указателя стека.

Регистры машины представлены в таблице 3.3.

Таблица 3.3 – Регистры машины

Регистр	Описание регистра
1	2
PC	Хранит адрес текущей выполняемой инструкции из памяти программы.
ACC	Хранит результат последней операции. Может быть любым объектом.
FRAME	Текущий кадр активации.
SP	Указатель стека.
CATCH-SP	Указатель стека для <code>catch</code> .

Сборка мусора включает в себя фазу пометки и фазу очистки. В фазе пометки помечаются объекты, до которых можно дойти с помощью обхо-

да, начиная с корневых объектов в регистрах. Это регистр аккумулятора АСС, указатель на текущий кадр активации FRAME, все объекты находящиеся в стеке, начиная с текущей позиции SP, а также все объекты в памяти констант и глобальных переменных.

Список инструкций представлен в таблице 3.4.

Таблица 3.4 – Инструкции виртуальной машины

Код	Мнемоника	Описание инструкции
1	2	3
0	CONST num	Поместить константу с номером num в регистр АСС.
1	JMP ofs	Безусловный переход на смещение ofs относительно PC.
2	JNT ofs	Если АСС == NIL, то относительный переход на смещение ofs.
3	ALLOC n	Создать новый кадр активации с числом аргументов n. Извлечь из стека аргументы начиная с позиции 1 (0-й элемент остается в стеке).
4	GLOBAL-REF i	Устанавливает регистру АСС значение глобальной переменной с индексом i.
5	GLOBAL-SET i	Устанавливает глобальной переменной с индексом i значение регистра АСС.
6	LOCAL-REF i	Загружает в АСС значение i-й локальной переменной (текущего кадра активации в окружении).
7	LOCAL-SET i	Присваивает i-й локальной переменной (текущего кадра активации в окружении) значение регистра АСС.
8	DEEP-REF i j	Загружает в АСС значение дальней переменной с индексом j в кадре i (начиная от текущего).
9	DEEP-SET i j	Присваивает дальней переменной j в кадре i значение регистра АСС.
10	PUSH	Добавляет значение регистра АСС в стек.

Продолжение таблицы 3.4

1	2	3
11	PACK n	Собирает последние n элементов из стека в список и добавляет этот список в стек.
12	REG-CALL ofs	Добавляет адрес следующей инструкции в стек и производит переход по смещению ofs.
13	RETURN	Производит переход на адрес из верхушки стека, при этом удаляет этот адрес из стека.
14	FIX-CLOSURE ofs	В регистр АСС записывается объект-замыкание с текущим кадром активации и смещением на код функции относительно текущего адреса ofs.
15	SAVE-FRAME	Сохраняет текущий кадр активации в стеке.
16	SET-FRAME num	Устанавливает кадр активации с номером num относительно начала глубины вызовов.
17	RESTORE-FRAME	Восстанавливает кадр активации из стека.
18	PRIM n	Вызывает примитив с номером n из таблицы примитивов с фиксированным числом аргументов.
19	NPRIM n	Вызывает примитив с номером n из таблицы примитивов с переменным числом аргументов.
20	HALT	Остановка машины.
21	PRIM-CLOSURE n	В регистр АСС записывается объект-замыкание с текущим кадром активации и адресом кода примитива с фиксированным числом аргументов и номером n.
22	NPRIM-CLOSURE n	В регистр АСС записывается объект-замыкание с текущим кадром активации и адресом кода примитива с переменным числом аргументов и номером n.

Продолжение таблицы 3.4

1	2	3
23	CATCH ofs	Добавляет запись в стек catch, сохраняется имя метки в асс, абсолютный адрес по смещению ofs, текущий кадр активации, указатель стека.
24	THROW	Извлекает из стека имя метки блока catch, ищет в стеке catch запись с этим именем (если не найдена – ошибка), восстанавливает кадр активации, указатель стека, выполняет переход на сохраненный адрес конца блока catch, отбрасывает все кадры стека catch выше найденного (включая его самого).
25	POP	Извлекает из стека значение и записывает в аккумулятор.
26	FUNC-CALL	Вызывает функцию-замыкание на вершине стека.
27	CHECK-PRIM	Является ли объект на вершине стека замыканием примитива.
28	PRIM-CALL	Вызывает функцию-замыкание как примитив на вершине стека.

В таблице 3.5 приведены коды примитивов, принимающих фиксированное число аргументов, а также число аргументов каждого примитива.

Таблица 3.5 – Примитивы с фиксированным числом аргументов

Код	Название примитива	Число фиксированных аргументов
1	2	3
1	car	1
2	cdr	1
3	atom	1
4	cons	2
5	rplaca	2
6	rplacd	2

Продолжение таблицы 3.5

1	2	3
7	%	2
8	<<	2
9	>>	2
10	eq	2
11	equal	2
12	>	2
13	<	2
14	sin	1
15	cos	1
16	sqrt	1
17	intern	1
18	symbol-name	1
19	symbol-function	1
20	string-size	1
21	inttostr	1
22	code-char	1
23	char-code	1
24	putchar	1
25	char	2
26	subseq	3
27	make-array	1
28	make-string	2
29	array-size	1
30	aref	2
31	seta	3
32	sets	3
33	symbolp	1
34	integerp	1
35	pairp	1
36	functionp	1
37	gensym	0
38	apply	2
39	round	1
40	~	1
41	+	2

Продолжение таблицы 3.5

1	2	3
42	-	2
43	*	2
44	/	2
45	&	2
46	bitor	2
47	^	2
48	arrayp	1
49	stringp	1
50	eval	1

В таблице 3.6 приведены коды примитивов, принимающих переменное число аргументов, а также число фиксированных аргументов каждого примитива.

Таблица 3.6 – Примитивы с переменным числом аргументов

Код	Название примитива	Число фиксированных аргументов
1	2	3
1	+	0
2	-	1
3	*	0
4	/	1
5	&	0
6	bitor	0
7	^	0
8	concat	0
9	funcall	1
10	print	0
11	error	0

4 Рабочий проект

В процессе разработки были выделены следующие модули системы: common, macro, compiler, optimizer, generator, assembler.

4.1 common.lsp

common.lsp — вспомогательный модуль, содержащий функции, являющиеся общими для нескольких других модулей.

Описание функций модуля common.lsp представлено в таблице 4.1.

Таблица 4.1 – Описание функций, используемых в модуле common.lsp

Название функции	Аргументы функции	Описание функции
1	2	3
comp-err	msg, &rest other	Вызывает исключение comp-err с сообщением msg и опциональной дополнительной информацией об ошибке other.
is-nary	args	Проверка на присутствие &rest аргумента в args.
mk/add-func	name list &rest other	Макрос для генерации функции name, добавляющей информацию компиляции функций в список list с дополнительной информацией other.
num-fix-args	list num	Определяет число фиксированных аргументов в списке list с начальным значением num.
remove-rest	list	Удаляет символ &rest из списка аргументов list.
make-nary-args	count args	Помещает необязательные аргументы из списка args, в котором count обязательных аргументов, в ещё один список.
search-symbol	list name	Поиск информации о функции или примитиве с названием name в списке list.
correct-lambda	f	Проверка на то, что S-выражение f является корректным лямбда-выражением.

4.2 macro.lsp

macro.lsp – модуль, отвечающий за регистрацию новых и раскрытие существующих макросов. Используется модулем compiler.lsp и преобразовывает S-выражения, осуществляя макроподстановку.

Описание функций модуля macro.lsp представлено в таблице 4.2.

Таблица 4.2 – Описание функций, используемых в модуле macro.lsp

Название функции	Аргументы функции	Описание функции
1	2	3
add-fix-macro	name env arity args body	Сгенерированная макросом mk/add-func функция для сохранения информации о макросе с фиксированным числом аргументов name с окружением env, арностью arity, параметрами args и телом body в список *fix-macros*.
add-nary-macro	name env arity args body	Сгенерированная макросом mk/add-func функция для сохранения информации о макросе с переменным числом аргументов name с окружением env, арностью arity, параметрами args и телом body в список *nary-macros*.
make-macro	list	Сохранить информацию о макросе. list – (name args body), где name – название макроса, args – список параметров макроса, body – тело макроса.
subst	sym env	Подставить значение символа или глобальной переменной sym с окружением env.
macro-eval	expr env	Рекурсивное раскрытие макроса expr с окружением env.
macro-eval-backquote	expr env	Макровычисление квазицитирования expr с окружением env.
macro-eval-if	expr env	Макровычисление условия expr с окружением env.

Продолжение таблицы 4.2

1	2	3
macro-eval-args	args env	Макровычисление списка аргументов args с окружением env.
macro-eval-progn	expr env	Макровычисление последовательности expr с окружением env.
extend-macro-env	env args vals	Расширение окружения подстановки макросов env символами args и их значениями vals.
macro-eval-let	args env	Макровычисление let-выражения args с временным расширением окружения env.
macro-eval-app-func	args body vals env	Применение пользовательской функции с параметрами args, телом body, аргументами vals и окружением env внутри макроса.
check-arguments	f type count args	Проверка на то, для функции или примитива f типа type список аргументов args имеет правильную длину в сравнении с числом фиксированных аргументов count.
macroexpand	args vals body	Раскрытие макроса с параметрами args, аргументами vals и телом body.
macro-eval-app	f args env	Макро применение функции, макроса или примитива f с аргументами args и окружением env.
macro-eval-funcall	f args env	Макровычисление вызова функции f с аргументами args и окружением env.
macro-eval-cond	list env	Макровычисление cond-выражения list с окружением env.
macro-eval-setq	var expr env	Присвоение макро символу var значения expr с окружением env.
macro-eval-function	s	Макровычисление функционального объекта s.
macro-expand-progn	expr env	Макро раскрытие последовательности expr с окружением env.

4.3 compiler.lsp

compiler.lsp – модуль, отвечающий за обработку исходного S-выражения и его преобразование в промежуточное представление, состоящее из более простых операций. Собирает вспомогательную информацию, такую как число использования тех или иных функций, наличие рекурсии в функциях и т.д., для дальнейшей оптимизации.

Описание функций модуля compiler.lsp представлено в таблице 4.3.

Таблица 4.3 – Описание функций, используемых в модуле compiler.lsp

Название функции	Аргументы функции	Описание функции
1	2	3
extend-env	env args	Расширяет окружение env новым кадром, состоящим из аргументов args.
add-func	name env arity args body	Сгенерированная макросом mk/add-func функция для сохранения информации о пользовательских функциях с фиксированным числом аргументов name с окружением env, арностью arity, параметрами args и телом body в список *fix-functions*.
add-nary-func	name env arity args body	Сгенерированная макросом mk/add-func функция для сохранения информации о пользовательских функциях с переменным числом аргументов name с окружением env, арностью arity, параметрами args и телом body в список *nary-functions*.
add-local-func	name env arity args body	Сгенерированная макросом mk/add-func функция для сохранения информации о локальных функциях name с окружением env, арностью arity и меткой real-name в список *local-functions*.

Продолжение таблицы 4.3

1	2	3
inner-compile	expr env tail	Рекурсивная функция компиляции выражения expr с окружением env в промежуточную форму. tail – является ли текущее выражение хвостовым.
compile-func-body	name args body env	Компиляция тела функции name с параметрами args, телом body и окружением env.
compile-lambda	name args body env	Регистрирует функцию name и компилирует её тело body. args – параметры функции, env – окружение.
find-func	f	Определение типа функции f и поиск её информации в соответствующем списке.
compile-progn	lst env tail	Компилирует блок progn со списком выражений lst и окружением env. tail – является ли текущее выражение хвостовым.
inc-function-ref	name	Увеличить число ссылок на функцию name
compile-application	f args env tail	Компиляция применения функции f с аргументами args в окружении env. tail – является ли текущее выражение хвостовым.
compile-function	f env	Компиляция создания замыкания функции f с окружением env.
compile-defun	expr env	Компилирует объявление функции expr с помощью defun в окружении env.
add-global	sym	Добавляет глобальную переменную sym и возвращает её индекс в списке глобальных переменных *global-variables*.
find-global-var	var	Производит поиск переменной в глобальном окружении по символу var. Возвращает индекс в списке глобального окружения, если символ найден, иначе nil.

Продолжение таблицы 4.3

1	2	3
var-search	params var	Вычисление порядкового номера переменной var в списке params с учетом &rest.
find-local-var	var env	Производит поиск переменной в локальном окружении env по символу var. Возвращает индекс переменной в окружении, если символ найден, иначе nil.
find-var	var env	Производит поиск переменной по символу var сначала в локальном окружении env, а затем в списке глобальных переменных *global-variables*. Возвращает список, состоящий из символа типа переменной (GLOBAL, LOCAL или DEEP) и индекса переменной.
compile-setq	body env	Компилирует setq-выражение с телом body и окружением env.
compile-if	expr env tail	Компилирует if-выражение с телом expr и окружением env. tail – является ли текущее выражение хвостовым.
compile-constant	c	Компиляция константы c.
compile-labels	lst env tail	Компилирует блок labels с телом lst и окружением env. tail – является ли текущее выражение хвостовым.
compile-variable	v env	Компиляция переменной v в окружении env.
compile-backquote	expr env	Компиляция квазичитирования с телом expr и окружением env.
compile-tagbody	body env	Компиляция tagbody с телом body и окружением env.
compile-catch	args env	Компиляция catch с телом args и окружением env.

Продолжение таблицы 4.3

1	2	3
compile-throw	args env	Компиляция throw с телом args и окружением env.
compile-defconst	body env tail	Компиляция defconst с телом body и окружением env и сохранение этой константы в хеше *const-vals*. tail – является ли текущее выражение хвостовым.
compile-apply	func args pack env tail	Компиляция применения функционального объекта func с аргументами args и окружением env. pack – нужно ли собирать аргументы в список (для примитива apply), tail – является ли текущее выражение хвостовым.
compile-args	args env	Компиляция аргументов args в окружении env.
compile	expr	Анализ S-выражения expr и преобразование в эквивалентное выражение, состоящее из более простых операций.

4.4 optimizer.lsp

optimizer.lsp – модуль, отвечающий за оптимизацию промежуточной формы. Осуществляет обход дерева в глубину.

Описание функций модуля optimizer.lsp представлено в таблице 4.4.

Таблица 4.4 – Описание функций, используемых в модуле optimizer.lsp

Название функции	Аргументы функции	Описание функции
1	2	3
accumulator-loading	tree	Удаляет избыточные формы загрузки значений в аккумулятор. tree – форма SEQ.

Продолжение таблицы 4.4

1	2	3
trivial-condition	alter	Если условие формы ALTER – константа, то считает значение условия и сокращает форму ALTER до одной из её веток. alter – форма ALTER.
simplify-arithmetic	tree	Заменяет вызов арифметического примитива tree с неограниченным количеством аргументов на вложенные вызовы примитивов с фиксированным количеством аргументов. tree – вызов примитива с переменным числом аргументов.
constant-folding	tree	Свёртка и распространение констант. tree – константа или вызов примитива.
expand-seqs	tree	Разворачивает вложенные SEQ. tree – форма SEQ.
dead-code-elimination	tree	Убирает из формы SEQ неиспользуемый код (последовательность команд загрузки аккумулятора (кроме последней команды и команды перед RETURN)).
tail-call	tree	Оптимизация хвостовой рекурсии. tree – форма TAIL-CALL или TAIL-NCALL.
unused-functions	tree	Удаление неиспользуемых функций. tree – форма FUNC.
search-abs-tree	exp nodes	Возвращает истину если найдены хотя бы один узел из списка nodes.
one-ref-args	body	Проверка на то, что в теле функции body к каждому аргументу только одно обращение.
search-closure-deep-ref	exp	Поиск всех FIX-CLOSURE и проверка каждого тела замыкания на наличие DEEP-REF.
beta-exp	exp args level	Выполнить подстановку тела функции exp с аргументами args и номером кадра активации level.

Продолжение таблицы 4.4

1	2	3
make-nary-param	num args	Создать выражение для подстановки аргументов args, num – число фиксированных аргументов.
beta-expansion	call	Подстановка тела функции call, когда функция вызывается один раз и это возможно.
can-inline	f	Возможно ли встраивание функции f.
side-effects	exp	Есть ли в выражении exp побочные эффекты.
optimize-tree1	tree	Первый проход оптимизации промежуточной формы tree. Возвращает оптимизированное дерево.
optimize-tree2	tree	Второй проход оптимизатора – встраивание функций, удаление после встраивания.
optimize-tree	tree	Оптимизация промежуточной формы tree.

4.5 generator.lsp

generator.lsp – модуль, отвечающий за генерацию линейного кода ассемблера, состоящего из меток и инструкций с мнемониками и операндами.

Описание функций модуля generator.lsp представлено в таблице 4.5.

Таблица 4.5 – Описание функций, используемых в модуле generator.lsp

Название функции	Аргументы функции	Описание функции
1	2	3
emit	val	Добавить инструкцию val в массив линейных инструкций *program*.
inner-generate	expr	Рекурсивная генерация кода для скомпилированного выражения expr.
generate-if	expr	Генерация ветвления. expr – список из условия, ветки по истине и ветки по лжи.

Продолжение таблицы 4.5

1	2	3
generate-2-params	expr	Генерация функции expr с 2-мя параметрами.
generate-deep	i j expr	Генерация формы DEEP-SET. i – смещение в окружении, j – индекс в кадре активации, expr – присваиваемое выражение.
generate-set	expr	Генерация форм GLOBAL-SET и LOCAL-SET.
generate-args	args set rev	Генерация вычисления аргументов args. set – тип инструкции для каждого аргумента (например, PUSH), rev – в обратном порядке.
generate-fix-prim	type args	Генерация вызова примитива с фиксированным числом аргументов type и аргументами args.
generate-nary-prim	type num args	Генерация вызова примитива с переменным числом аргументов type и аргументами args. num – число фиксированных аргументов.
generate-reg-call	name fix- num env args	Генерация обычного вызова функции name с аргументами args и окружением env. fix-num – число фиксированных аргументов для функций с переменным числом аргументов.
generate-let	count args body	Генерация let-формы с телом body и аргументами args, расширение окружения env.
generate-closure	op name env body	Генерация замыкания. op – инструкция создания замыкания (PRIM-CLOSURE, NPRIM-CLOSURE или FIX-CLOSURE), name – название функционального объекта, env – окружение, body – опциональное тело (для лямбда-выражения).
generate-catch	tag body	Генерация формы catch с меткой tag и телом body.
generate-throw	tag res	Генерация формы throw с меткой tag и значением res.

Продолжение таблицы 4.5

1	2	3
generate-tail-call	name fix-num depth args	Генерация хвостового вызова функции name с числом фиксированных аргументов fix-num, глубиной вызова относительно входа в функцию depth и аргументами args.
generate-nary	args	Генерация списка необязательных аргументов args.
generate-apply	func args pack	Генерация применения функции func к аргументам args. pack – нужно ли объединять аргументы в список (для примитива apply).
generate	expr	Генерация кода для скомпилированного выражения expr.

4.6 assembler.lsp

assembler.lsp – модуль, отвечающий за ассемблирование линейного кода в байт-код виртуальной машины, представляющий собой массив байт.

Описание функций модуля assembler.lsp представлено в таблице 4.6.

Таблица 4.6 – Описание функций, используемых в модуле assembler.lsp

Название функции	Аргументы функции	Описание функции
1	2	3
last-cdr	list	Найти последний cdr в списке list.
asm-emit	&rest bytes	Записать байты bytes в список bytecode.
assemble-label	inst	Ассемблирование метки inst.
assemble-const	inst	Ассемблирование инструкции загрузки константы inst в ACC.
assemble-prim	inst	Ассемблирование инструкции вызова примитива inst.

Продолжение таблицы 4.6

1	2	3
assemble- inst	inst jmp- labels	Ассемблирование инструкции общего вида inst.
assemble- first-pass	program	Первый проход ассемблера (замена мнемоник инструкций соответствующими байтами и сбор информации о метках и переходах).
assemble- second-pass	first-pass- res	Второй проход ассемблера (замена меток переходов на относительные адреса).
assemble	program	Преобразует список инструкций с метками program в байт-код.

4.7 Модульное тестирование фазы компиляции

Модульные тесты фазы компиляции представлены в таблице 4.7.

Таблица 4.7 – Модульные тесты компилятора

Описание теста	Исходное S-выражение	Ожидаемый результат компиляции
1	2	3
Компиляция progn	(progn 1 2 3)	(SEQ (CONST 1) (SEQ (CONST 2) (CONST 3)))
Компиляция if	(if t (progn 1 2) 3)	(ALTER (GLOBAL-REF 0) (SEQ (CONST 1) (CONST 2)) (CONST 3))
Компиляция setq	(setq a (* 2 3))	(GLOBAL-SET 2 (NARY-PRIM * 0 ((CONST 2) (CONST 3))))
Компиляция объявления и вызова функции	(progn (defun f (x) x) (f 5))	(SEQ (FUNC F (SEQ (LOCAL-REF 0) (RETURN))) (FIX-CALL F 0 ((CONST 5)) ()))

Продолжение таблицы 4.7

1	2	3
Компиляция let и обращения к дальней переменной	((lambda (x) ((lambda (y) (print x y)) 10)) 5)	(FIX-LET 1 ((CONST 5)) (FIX-LET 1 ((CONST 10)) (NARY-PRIM PRINT 0 ((DEEP-REF 1 0) (LOCAL-REF 0)))))
Компиляция вызова функционального объекта	(funcall #' + 1 2)	(APPLY (NPRIM-CLOSURE +) ((CONST 1) (CONST 2)) T)
Компиляция tagbody	(tagbody (go end) 5 end)	(SEQ (GOTO END) (SEQ (CONST 5) (SEQ (LABEL END) (CONST NIL)))))
Компиляция backquote	(progn (setq a 5) `(a = ,a))	(SEQ (GLOBAL-SET 2 (CONST 5)) (FIX-PRIM CONS ((CONST A) (FIX-PRIM CONS ((CONST =) (FIX-PRIM CONS ((GLOBAL-REF 2) (CONST ())))))))))
Компиляция catch и throw	(catch 'err (throw 'err "err"))	(CATCH (CONST ERR) (THROW (CONST ERR) (CONST "err")))

4.8 Модульное тестирование фазы оптимизации

Модульные тесты фазы оптимизации представлены в таблице 4.8.

Таблица 4.8 – Модульные тесты оптимизатора

Описание теста	Исходное выражение	Ожидаемое оптимизированное выражение
1	2	3
Упрощение тривиального условия	(ALTER (CONST T) (CONST 1) (CONST 2))	(CONST 1)
Оптимизация арифметического выражения	(NARY-PRIM + 0 ((LOCAL-REF 0) (LOCAL-REF 1)))	(FIX-PRIM + ((LOCAL-REF 0) (LOCAL-REF 1)))
Оптимизация загрузки аккумулятора	(SEQ (CONST 1) (CONST 2) (GLOBAL-SET 2 (CONST 3)) (CONST 4) (GLOBAL-REF 2))	(SEQ (GLOBAL-SET 2 (CONST 3)))
Свёртка констант	(GLOBAL-SET 2 (FIX-PRIM + ((CONST 1) (CONST 2))))	(GLOBAL-SET 2 (CONST 3))
Встраивание и удаление функции	(SEQ (FUNC LIST (SEQ (SEQ (CONST "Функция создания списка") (LOCAL-REF 0)) (RETURN))) (NARY-CALL LIST 0 0 ((CONST 1) (CONST 2) (CONST 3)) ()))	(NARY ((CONST 1) (CONST 2) (CONST 3)))
Встраивание функции и удаление неиспользуемых функций	(SEQ (FUNC F1 (SEQ (CONST 10) (RETURN))) (FUNC F2 (SEQ (CONST 10) (RETURN))) (FIX-CALL F2 0 () ()))	(CONST 10)

Продолжение таблицы 4.8

1	2	3
Встраивание функций, удаление неиспользуемых функций и свёртка констант	(SEQ (FUNC LIST (SEQ (SEQ (CONST "Функция создания списка") (LOCAL-REF 0)) (RETURN))) (SEQ (FUNC F (SEQ (NARY-PRIM + 0 ((NARY-PRIM * 0 ((FIX-PRIM CAR ((NARY-CALL LIST 0 0 ((CONST 1) (CONST 2) (CONST 3)) ()))) (CONST 2))) (LOCAL-REF 0) (CONST 10))) (RETURN))) (FIX-CALL F 0 ((CONST 5)) ()))))	(CONST 17)

4.9 Системное тестирование разработанного компилятора

Было проведено системное тестирование разработанного компилятора, покрывающее все фазы компиляции и проверяющее корректность работы скомпилированного кода на тестах библиотек Common Lisp, исчерпывающе покрывающих основные граничные случаи компилятора. Успешным результатом данного тестирования является факт успешного выполнения тестов библиотек на виртуальной машине с помощью байт-кода, сгенерированного компилятором.

4.9.1 Технология системного тестирования

Для тестирования написана программа на языке C, реализующая виртуальную машину и читающая файл из стандартного потока ввода. Формат

входного файла должен включать следующие элементы, разделённые пробелами:

- размер байт-кода N (суммарное количество опкодов и операндов);
- непосредственно байт-код размером N (N чисел);
- массив констант (например, #(T () 0 1 ...));
- размер массива глобальных переменных;
- хеш-таблица меток и их адресов (например, ((ABS . 2) (G314 . 22) (NOT . 27) ...)).

Пример файла скомпилированной программы (print 5):

```
8 0 2 10 11 1 19 9 20 #( T () 5 ) 2 ()
```

Здесь 8 – размер байт-кода, затем идёт сам байт-код, представляющий собой 8 чисел, потом массив констант – #(T () 5), размер массива глобальных переменных – 2 и хеш-таблица меток и адресов – (), в данном случае пустая. Таким образом, программа на C загружает все данные из файла, инициализирует состояние виртуальной машины и запускает выполнение байт-кода.

Для удобного автоматического тестирования скомпилирована версия самого компилятора, который загружает S-выражение из массива констант и выполняет дальнейшую компиляцию и печать бинарных данных в выше указанном формате. Константа исходного S-выражения в бинарном файле компилятора имеет следующий вид:

```
(PROGN *TARGET*)
```

При подстановке S-выражения, которое требуется скомпилировать, вместо слова *TARGET*, и выполнения этого бинарного файла на виртуальной машине на выходе появляется бинарный файл скомпилированного исходного S-выражения. Если теперь этот файл выполнить на виртуальной машине, то произойдёт выполнение исходного S-выражения.

С помощью программных утилит, например coreutils (cat, sed), и перенаправления стандартных потоков ввода и вывода можно автоматизировать

весь этот процесс компиляции произвольных программ на Common Lisp и их выполнения на виртуальной машине с помощью, например, скриптов bash или Makefile.

4.9.2 Тестирование «AES»

aes – библиотека, реализующая алгоритм AES для шифрования и дешифрования данных. В ходе тестирования проверяются:

- процедура расширения ключа для AES-128;
- шифрование и дешифрование AES-128 без дополнения;
- шифрование и дешифрование с дополнением по схеме CMS;
- корректность формирования и удаления CMS-дополнения при шифровании.

Результаты проведения тестирования на виртуальной машине представлены на рисунке 4.1.

```
1 "Тестирование расширения ключа AES-128"
2 (OK #(#(0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15) #(214 170 116 253 210 175 114
   250 218 166 120 241 214 171 118 254) #(182 146 207 11 100 61 189 241 190
   155 197 0 104 48 179 254) #(182 255 116 78 210 194 201 191 108 89 12 191 4
   105 191 65) #(71 247 247 188 149 53 62 3 249 108 50 188 253 5 141 253)
   ...) #(#(0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15) #(214 170 116 253 210 175
   114 250 218 166 120 241 214 171 118 254) #(182 146 207 11 100 61 189 241
   190 155 197 0 104 48 179 254) #(182 255 116 78 210 194 201 191 108 89 12
   191 4 105 191 65) #(71 247 247 188 149 53 62 3 249 108 50 188 253 5 141
   253) ...))
3 "Тесты для AES-128 шифрования и дешифрования"
4 (OK #(15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0) #(15 14 13 12 11 10 9 8 7 6 5 4
   3 2 1 0))
5 (OK #(11 14 12 9 1 6 8 13 3 15 2 0 10 7 5 4) #(11 14 12 9 1 6 8 13 3 15 2 0
   10 7 5 4))
6 "Тестирование шифрования и дешифрования данных с паддингом CMS"
7 (OK #(1 2 3 4 5 6 7) #(1 2 3 4 5 6 7))
8 (OK #(0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19) #(0 1 2 3 4 5 6 7 8
   9 10 11 12 13 14 15 16 17 18 19))
9 "Тест правильности паддинга CMS при шифровании данных"
10 (OK #(10 20 30 40 50) #(10 20 30 40 50))
11 "Успешно завершено тестов: " 6 / 6
12 "Все тесты завершены успешно"
```

Рисунок 4.1 – Результаты выполнения тестов AES на виртуальной машине

4.9.3 Тестирование «regex»

regex – библиотека регулярных выражений. В ходе тестирования проверяется парсинг regex-шаблонов и множество случаев поиска regex-шаблона в строке.

Результаты проведения тестирования на виртуальной машине представлены на рисунке 4.2.

```
1 "Тестирование символа"
2 (OK (SYM #\a) (SYM #\a))
3 "Тестирование количества"
4 (OK (REPEAT 12 ()) (REPEAT 12 ()))
5 (OK (REPEAT 12 MORE) (REPEAT 12 MORE))
6 (OK (REPEAT 12 50) (REPEAT 12 50))
7 "Тестирование количества"
8 (OK (RANGE () ((65 ()) (CLASS DIGIT) (CLASS NSPACE) (45 ()) (98 ()))) (RANGE
9   () ((65 ()) (CLASS DIGIT) (CLASS NSPACE) (45 ()) (98 ())))
10 (OK (RANGE NEGATIVE ((65 ()))) (RANGE NEGATIVE ((65 ())))
11 (OK (RANGE () ((65 122) (66 115))) (RANGE () ((65 122) (66 115))))
12 "Тестирование квантификатора"
13 (OK ((REPEAT 12 ()) LAZY) ((REPEAT 12 ()) LAZY))
14 (OK ((PLUS) LAZY) ((PLUS) LAZY))
15 (OK ((STAR) LAZY) ((STAR) LAZY))
16 "Тестирование элемента"
17 (OK (LAZY-STAR (SYM #\+)) (LAZY-STAR (SYM #\+)))
18 (OK (OR (EPSILON) (SYM #\b)) (OR (EPSILON) (SYM #\b)))
19 (OK (SEQ (SYM #\b) (STAR (...))) (SEQ (SYM #\b) (STAR (SYM #\b)))))
20 (OK (LAZY-STAR (ANY)) (LAZY-STAR (ANY)))
21 ...
22 "Тестирование регулярных выражений"
23 (OK "" "" ) STRING "a" REGEX "a{0}" GROUPS ( )
24 (OK "abcccd" "abcccd" ) STRING "abcccd" REGEX "abc{4}d" GROUPS ( )
25 (OK "assaaabbaa" "assaaabbaa" ) STRING "assaaabbaa" REGEX "a+(s*)(a+(b+)a+)"
26   GROUPS ("ss" "aaabbaa" "bb")
27 ...
28 (OK "привет" "привет" ) STRING "abc_123привет" REGEX "[^A-Za-z0-9_]+" GROUPS
29   ( )
30 (OK ".*?" ".*?" ) STRING ".*?" REGEX "\.\\*\\?" GROUPS ( )
31 (OK "nonnoncapturinggroups" "nonnoncapturinggroups" ) STRING "
32   nonnoncapturinggroups" REGEX "(?:non)+(?:capturing)(?:groups)" GROUPS ( )
33 (OK "134" "134" ) STRING "_134_" REGEX "(1)(2|34|9)" GROUPS ("1" "34")
34 (OK "1345" "1345" ) STRING "_1345_" REGEX "(1)(34|3|2)(45)" GROUPS ("1" "3" "
35   45")
36 ...
37 "Успешно завершено тестов: " 76 / 76
38 "Все тесты завершены успешно"
```

Рисунок 4.2 – Результаты выполнения тестов regex на виртуальной машине

4.9.4 Тестирование «PNG»

png – библиотека для парсинга файлов изображений в формате PNG. В ходе тестирования проверяется успешный разбор PNG-файла.

Результаты проведения тестирования на виртуальной машине представлены на рисунке 4.3.

```
1 (ОК ##(##(145 99 170 255) #(119 85 177 255) #(189 75 108 255) #(251 186 181
2 255) #(69 117 255 255) #(176 189 168 255) #(255 235 59 255) #(255 235 59
3 255)) ##(41 98 255 255) #(118 117 163 255) #(42 98 254 255) #(97 165 154
4 255) #(64 148 146 255) #(193 218 108 255) #(255 237 74 255) #(255 237 78
5 255)) ##(41 98 255 255) #(229 146 31 255) #(73 105 217 255) #(42 99 253
6 255) #(76 175 80 255) #(103 187 106 255) #(150 178 255 255) #(66 115 255
7 255)) ##(90 134 255 255) #(235 150 32 255) #(99 129 223 255) #(100 162
8 254 255) #(128 216 255 255) #(128 216 255 255) #(119 205 255 255) #(96 172
9 255 255)) ##(131 217 255 255) #(241 159 28 255) #(158 201 195 255) #(137
10 218 240 255) #(160 221 206 255) #(160 221 206 255) #(160 221 206 255)
11 #(160 221 206 255)) ##(149 222 255 255) #(241 161 32 255) #(178 218 210
12 255) #(254 236 64 255) #(255 235 59 255) #(255 235 59 255) #(255 235 59
13 255) #(255 235 59 255)) ##(##(145 99 170 255) #(119 85 177 255) #(189 75
14 108 255) #(251 186 181 255) #(69 117 255 255) #(176 189 168 255) #(255 235
15 59 255) #(255 235 59 255)) ##(41 98 255 255) #(118 117 163 255) #(42 98
16 254 255) #(97 165 154 255) #(64 148 146 255) #(193 218 108 255) #(255 237
17 74 255) #(255 237 78 255)) ##(41 98 255 255) #(229 146 31 255) #(73 105
18 217 255) #(42 99 253 255) #(76 175 80 255) #(103 187 106 255) #(150 178
19 255 255) #(66 115 255 255)) ##(90 134 255 255) #(235 150 32 255) #(99 129
20 223 255) #(100 162 254 255) #(128 216 255 255) #(128 216 255 255) #(119
21 205 255 255) #(96 172 255 255)) ##(131 217 255 255) #(241 159 28 255)
22 #(158 201 195 255) #(137 218 240 255) #(160 221 206 255) #(160 221 206
23 255) #(160 221 206 255) #(160 221 206 255)) ##(149 222 255 255) #(241 161
24 32 255) #(178 218 210 255) #(254 236 64 255) #(255 235 59 255) #(255 235
25 59 255) #(255 235 59 255) #(255 235 59 255)))
26 ()
27 "Успешно завершено тестов: " 1 / 1
28 "Все тесты завершены успешно"
```

Рисунок 4.3 – Результаты выполнения тестов PNG на виртуальной машине

4.9.5 Тестирование «JPEG»

jpeg – библиотека для парсинга файлов изображений в формате JPEG. В ходе тестирования проверяется успешный разбор JPEG-файла.

Результаты проведения тестирования на виртуальной машине представлены на рисунке 4.4.

```

1 P3 113 115 255
2 #(#(0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0
  87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0
  87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87
  36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87
  36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36
  0 87 36 0 87 36 0 90 36 0 89 36 0 88 36 0 87 36 3 85 36 6 84 36 10 82 36
  10 82 36 13 80 36 12 81 36 10 82 36 9 82 36 7 83 36 6 84 36 5 84 36 3 85
  36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36
  0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0
  87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0
  87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87
  36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87
  36 0 87 36 0 87 36 0 87 36) #(0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87
  36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87
  36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36
  0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0
  87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0
  87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 89 36 0 89 36 0 88 36 0 87
  36 3 85 36 6 84 36 9 82 36 9 82 36 12 81 36 10 82 36 9 82 36 7 83 36 6 84
  36 5 84 36 3 85 36 2 86 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87
  36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 0 87 36 ...

```

Рисунок 4.4 – Результаты выполнения тестов JPEG на виртуальной машине

4.9.6 Тестирование «Lua»

lua – библиотека для трансляции Lua-скриптов в Common Lisp. В ходе тестирования проверяется лексический, синтаксический анализ и трансляция программы на языке Lua в аналогичную программу на Common Lisp.

Результаты проведения тестирования на виртуальной машине представлены на рисунке 4.5.

```

1 "Выполняет тест парсинга лексемы переменной"
2 (OK ABCDF ABCDF)
3 "Выполняет тест парсинга лексемы числа в шестнадцетиричном виде"
4 (OK 175 175)
5 "Выполняет тест парсинга лексемы числа в шестнадцетиричном виде"
6 (OK 175 175)
7 "hash test"
8 2 2
9 (HASH ("a" "first") (1 "second") (2 "third") ("b" "fourth")))
10 "Выполняет тест парса функции"
11 (FUNCTION (LAMBDA (ARG) (PROGN (LUA-SET (ARG) (100)))))
12 "Выполняет тест лексического на Lua в Lisp-выражение"
13 (PRINT #\ ( A #\ . B #\ . C #\ ) A LUA-SET #\{ B LUA-SET "1" #\ , C LUA-SET "2"
    #\ , 3 #\ , 4 #\ } #\ ; ABC LUA-SET 100 END B LUA-SET "aboaasd" C LUA-SET
    FALSE-CONST LUA-AND NIL-CONST)
14 "Выполняет тест преобразования строки на Lua в Lisp-выражение"
15 (PROGN (LUA-SET (PERSON) ((LUA-CREATETABLE ()))) (LUA-SET-INDEX PERSON "NEW"
    (FUNCTION (LAMBDA (NAME AGE) (PROGN (LET ((P (LUA-CREATETABLE ()))) (PROGN
    (LUA-SET ((LUA-GET-INDEX P "NAME")) (NAME)) (LUA-SET ((LUA-GET-INDEX P "
    AGE")) (AGE)) (LUA-SET-INDEX P "GET_NAME" (FUNCTION (LAMBDA (SELF) (PROGN
    (LUA-GET-INDEX SELF "NAME"))))) (LUA-SET-INDEX P "SET_NAME" (FUNCTION (
    LAMBDA (SELF NAME) (PROGN (LUA-SET ((LUA-GET-INDEX SELF "NAME")) (NAME))))
    )) (LUA-SET-INDEX P "HELLO" (FUNCTION (LAMBDA (SELF) (PROGN (LUA-CONCAT (
    LUA-CONCAT (LUA-CONCAT (LUA-CONCAT "Hello , my name is " (LUA-GET-INDEX
    SELF "NAME")) " , i am " ) (LUA-GET-INDEX SELF "AGE")) " years old"))))) P))
    )))) (LUA-SET (VASYA) ((FUNCALL (LUA-GET-INDEX PERSON "NEW") "Vasya" 30)))
    (LUA-SET (KOLYA) ((FUNCALL (LUA-GET-INDEX PERSON "NEW") "Kolya" 20))) (
    LUA-SET (VASYA KOLYA) (KOLYA VASYA)) (FUNCALL PRINT (LET ((__T VASYA)) (
    FUNCALL (LUA-GET-INDEX __T "HELLO") __T))) (FUNCALL PRINT (LET ((__T KOLYA
    )) (FUNCALL (LUA-GET-INDEX __T "HELLO") __T))))))
16 "Выполняет тест исполнения Lisp-выражения, преобразованного из Lua кода"
17 (PROGN (LUA-SET (PERSON) ((LUA-CREATETABLE ()))) (LUA-SET-INDEX PERSON "NEW"
    (FUNCTION (LAMBDA (NAME AGE) (PROGN (LET ((P (LUA-CREATETABLE ()))) (PROGN
    (LUA-SET ((LUA-GET-INDEX P "NAME")) (NAME)) (LUA-SET ((LUA-GET-INDEX P "
    AGE")) (AGE)) (LUA-SET-INDEX P "GET_NAME" (FUNCTION (LAMBDA (SELF) (PROGN
    (LUA-GET-INDEX SELF "NAME"))))) (LUA-SET-INDEX P "SET_NAME" (FUNCTION (
    LAMBDA (SELF NAME) (PROGN (LUA-SET ((LUA-GET-INDEX SELF "NAME")) (NAME))))
    )) (LUA-SET-INDEX P "HELLO" (FUNCTION (LAMBDA (SELF) (PROGN (LUA-CONCAT (
    LUA-CONCAT (LUA-CONCAT (LUA-CONCAT "Hello , my name is " (LUA-GET-INDEX
    SELF "NAME")) " , i am " ) (LUA-GET-INDEX SELF "AGE")) " years old"))))) P))
    )))) (LUA-SET (VASYA) ((FUNCALL (LUA-GET-INDEX PERSON "NEW") "Vasya" 30)))
    (LUA-SET (KOLYA) ((FUNCALL (LUA-GET-INDEX PERSON "NEW") "Kolya" 20))) (
    LUA-SET (VASYA KOLYA) (KOLYA VASYA)) (FUNCALL PRINT (LET ((__T VASYA)) (
    FUNCALL (LUA-GET-INDEX __T "HELLO") __T))) (FUNCALL PRINT (LET ((__T KOLYA
    )) (FUNCALL (LUA-GET-INDEX __T "HELLO") __T))))))
18 "Успешно завершено тестов: " 3 / 3
19 "Все тесты завершены успешно"

```

Рисунок 4.5 – Результаты выполнения тестов Lua на виртуальной машине

ЗАКЛЮЧЕНИЕ

Разработанная программная система оптимизирующего компилятора Common Lisp представляет собой полностью функциональный транслятор исходного кода на языке Common Lisp в байт-код, реализующий все основные возможности языка, а также виртуальную машину для выполнения исходных программ.

Основные результаты работы:

1. Изучен процесс компиляции языков программирования и функциональных языков в частности. Сформулированы основные стадии компиляции.

2. Изучены методы оптимизации компиляторов. Подобран список оптимизаций, которые возможно реализовать при компиляции Common Lisp.

3. Спроектирован компилятор с оптимизациями. Разработана архитектура компилятора и программный интерфейс. Составлен список реализуемых оптимизаций.

4. Реализована компиляция S-выражения в промежуточную форму, состоящую из простых операций и сохраняющую смысл исходного выражения, а также сбор информации о функциях для их дальнейшей оптимизации.

5. Реализована генерация кода ассемблера. Для этого происходит обход дерева промежуточной формы в глубины и для каждой операции определённым образом генерируется набор линейных инструкций ассемблера, состоящих из мнемоник и операндов, а также меток для ветвления и функций.

6. Реализована генерация байт-кода. Происходит в 2 прохода. В первый проход в каждой инструкции мнемоника заменяется на соответствующий опкод, константы собираются в список без дубликатов, а обращения к этим константам содержат индексы в этом списке, для меток запоминается их адрес в байт-коде, инструкции переходов и вызова функции сохраняются для дальнейшей замены меток. Во втором проходе все упоминания меток заменяются на относительные адреса, полученные в первом проходе.

7. Реализована оптимизация промежуточной формы. Промежуточное дерево, получившееся в результате статического анализа, преобразуется посредством применения оптимизаций. Происходит полный обход дерева в глубину, где для разных операций предусмотрены различные виды оптимизаций, и вызываются соответствующие функции.

Все требования, объявленные в техническом задании, были полностью реализованы, все задачи, поставленные в начале разработки проекта, были также решены.

Готовый рабочий проект представлен в виде программной системы компилятора Common Lisp. Исходный код находится в публичном доступе и внедрён в os-рі.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Abelson, H. Structure and Interpretation of Computer Programs / H. Abelson, G. J. Sussman, J. Sussman. – Cambridge : The MIT Press, 1996. – 657 с. – ISBN 978-0-262-01153-2. – Текст : непосредственный.
2. Queinnec, C. Lisp in Small Pieces / C. Queinnec. – Cambridge : Cambridge University Press, 1996. – 534 с. – ISBN 978-0-521-56247-8. – Текст : непосредственный.
3. Aho, A. V. Compilers: Principles, Techniques, and Tools, Updated 2E, 1/E / A. V. Aho. – Noida : PEARSON INDIA, 2023. – 224 с. – ISBN 978-93-5705-411-9. – Текст : непосредственный.
4. Cooper, K. D. Engineering a Compiler / K. D. Cooper, L. Torczon. – Amsterdam : Elsevier Science, 2022. – 848 с. – ISBN 978-0-12-815412-0. – Текст : непосредственный.
5. Appel, A. W. Modern Compiler Implementation in C / A. W. Appel, J. Palsberg. – Cambridge : Cambridge University Press, 2000. – 544 с. – ISBN 978-81-7596-071-8. – Текст : непосредственный.
6. Steele, G. Common LISP: The Language / G. Steele. – Amsterdam : Elsevier S & T, 1990. – 1056 с. – ISBN 978-0-08-050226-7. – Текст : непосредственный.
7. Graham, P. ANSI Common LISP / P. Graham. – Upper Saddle River : Pearson, 1995. – 448 с. – ISBN 978-0-13-370875-2. – Текст : непосредственный.
8. Seibel, P. Practical Common Lisp / P. Seibel. – New York : Apress, 2012. – 500 с. – ISBN 978-1-4302-4290-1. – Текст : непосредственный.
9. Graham, P. On Lisp: Advanced Techniques for Common Lisp / P. Graham. – Hoboken : Prentice Hall, 1993. – 413 с. – ISBN 978-0-13-030552-7. – Текст : непосредственный.
10. Hoyte, D. Let Over Lambda 50 Years of Lisp / D. Hoyte. – Morrisville : Lulu.com, 2008. – 384 с. – ISBN 978-1-4357-1275-1. – Текст : непосредственный.

11. Hekmatpour, S. LISP A Portable Implementation / S. Hekmatpour. – Englewood Cliffs : Prentice Hall, 1989. – 177 с. – ISBN 978-0-13-537490-0. – Текст : непосредственный.
12. Levin, M. I. LISP 1.5 Programmer's Manual / M. I. Levin. – Cambridge : The MIT Press, 1962. – 112 с. – ISBN 978-0-262-13011-0. – Текст : непосредственный.
13. Allen, J. Anatomy of Lisp / J. Allen. – New York : McGraw-Hill College, 1978. – 464 с. – ISBN 978-0-07-001115-1. – Текст : непосредственный.
14. Holden, D. Build Your Own Lisp / D. Holden. – North Charleston : CreateSpace Independent Publishing Platform, 2014. – 213 с. – ISBN 978-1-5010-0662-3. – Текст : непосредственный.
15. Holm, N. M. LISP from Nothing / N. M. Holm. – Morrisville : Lulu Press, 2020. – 351 с. – ISBN 978-1-7165-0883-7. – Текст : непосредственный.
16. Muchnick, S. Advanced Compiler Design Implementation / S. Muchnick. – San Francisco : Morgan Kaufmann, 1997. – 888 с. – ISBN 978-1-55860-320-2. – Текст : непосредственный.
17. Allen, R. Optimizing Compilers for Modern Architectures: A Dependence-based Approach / R. Allen, K. Kennedy. – San Francisco : Morgan Kaufmann, 2001. – 816 с. – ISBN 978-1-55860-286-1. – Текст : непосредственный.
18. Morgan, R. Building an Optimizing Compiler / R. Morgan. – Amsterdam : Elsevier Science & Technology Books, 1998. – 472 с. – ISBN 978-1-55558-179-4. – Текст : непосредственный.
19. Pande, S. Compiler Optimizations for Scalable Parallel Systems Languages, Compilation Techniques, and Run Time Systems / S. Pande. – Berlin : Springer Science & Business Media, 2001. – 812 с. – ISBN 978-3-540-41945-7. – Текст : непосредственный.
20. Loudon, K. C. Compiler Construction: Principles and Practice / K. C. Loudon. – Boston : Cengage Learning, 1997. – 592 с. – ISBN 978-0-534-93972-4. – Текст : непосредственный.

21. Holub, A. I. Compiler design in C / A. I. Holub. – Englewood Cliffs : Prentice-Hall, 1990. – 924 с. – ISBN 978-0-13-155045-2. – Текст : непосредственный.
22. Sandler, N. Writing a C Compiler Build a Real Programming Language from Scratch / N. Sandler. – San Francisco : No Starch Press, 2024. – 792 с. – ISBN 978-1-7185-0042-6. – Текст : непосредственный.
23. Nystrom, R. Crafting Interpreters / R. Nystrom. – [б. м.] : Genever Benning, 2021. – 639 с. – ISBN 978-0-9905829-3-9. – Текст : непосредственный.
24. Colombet, Q. LLVM Code Generation A Deep Dive Into Compiler Backend Development / Q. Colombet. – Birmingham : Packt Publishing, 2025. – 608 с. – ISBN 978-1-83763-778-2. – Текст : непосредственный.
25. Levine, J. R. Linkers and Loaders / J. R. Levine. – San Francisco : Morgan Kaufmann, 2000. – 272 с. – ISBN 978-1-55860-496-4. – Текст : непосредственный.
26. Pierce, B. C. Types and Programming Languages / B. C. Pierce. – Cambridge : The MIT Press, 2002. – 648 с. – ISBN 978-0-262-16209-8. – Текст : непосредственный.
27. Siek, J. G. Essentials of Compilation An Incremental Approach in Racket / J. G. Siek. – Cambridge : The MIT Press, 2023. – 240 с. – ISBN 978-0-262-04776-0. – Текст : непосредственный.
28. Skeppstedt, J. An Introduction to the Theory of Optimizing Compilers With Performance Measurements on POWER / J. Skeppstedt. – North Charleston : CreateSpace Independent Publishing Platform, 2018. – 289 с. – ISBN 978-1-7259-3048-3. – Текст : непосредственный.
29. Srikant, Y. N. The Compiler Design Handbook Optimizations and Machine Code Generation, Second Edition / Y. N. Srikant, P. Shankar. – London : Taylor & Francis, 2007. – 794 с. – ISBN 978-1-4200-4382-2. – Текст : непосредственный.

30. Nisan, N. The Elements of Computing Systems Building a Modern Computer from First Principles / N. Nisan, S. Schocken. – Cambridge : The MIT Press, 2008. – 342 с. – ISBN 978-0-262-64068-8. – Текст : непосредственный.
31. Gabriel, R. P. Performance and Evaluation of Lisp Systems / R. P. Gabriel. – Cambridge : The MIT Press, 1985. – 285 с. – ISBN 978-0-262-07093-5. – Текст : непосредственный.
32. Brooks, R. A. Programming in Common LISP / R. A. Brooks. – New York : Wiley, 1985. – 303 с. – ISBN 978-0-471-81888-5. – Текст : непосредственный.
33. Winston, P. H. Lisp / P. H. Winston, B. K. Horn. – Hoboken : Prentice Hall, 1988. – 611 с. – ISBN 978-0-201-08319-4. – Текст : непосредственный.
34. Touretzky, D. S. Common LISP: A Gentle Introduction to Symbolic Computation / D. S. Touretzky. – Mineola : Dover Publications, 2013. – 608 с. – ISBN 978-0-486-49820-1. – Текст : непосредственный.
35. Friedman, D. P. The Little LISPer, Third Edition / D. P. Friedman. – Upper Saddle River : Pearson, 1989. – 222 с. – ISBN 978-0-02-339763-9. – Текст : непосредственный.
36. Barski, C. Land of Lisp: Learn to Program in Lisp, One Game at a Time! / C. Barski. – New York : Random House Publishing Services, 2010. – 800 с. – ISBN 978-1-59327-349-1. – Текст : непосредственный.
37. Yuasa, T. Introduction to Common Lisp / T. Yuasa, M. Hagiya. – Burlington : Morgan Kaufmann Publishers, 1987. – 293 с. – ISBN 978-0-12-774860-3. – Текст : непосредственный.
38. Sangal, R. Programming Paradigms in Lisp / R. Sangal. – New York : McGraw-Hill College, 1991. – 384 с. – ISBN 978-0-07-054666-0. – Текст : непосредственный.
39. Slade, S. Object-Oriented Common Lisp / S. Slade. – Englewood Cliffs : Prentice Hall, 1997. – 774 с. – ISBN 978-0-13-605940-0. – Текст : непосредственный.
40. Keene, S. E. Object-Oriented Programming in COMMON LISP: A Programmer's Guide to CLOS / S. E. Keene. – Reading : Addison-Wesley

Professional, 1989. – 288 с. – ISBN 978-0-201-17589-9. – Текст : непосредственный.

41. Watson, M. Common LISP Modules / M. Watson. – New York : Springer, 1991. – 207 с. – ISBN 978-0-387-97614-3. – Текст : непосредственный.

42. Domkin, V. Programming Algorithms in Lisp: Writing Efficient Programs with Examples in ANSI Common Lisp / V. Domkin. – New York : Apress, 2021. – 392 с. – ISBN 978-1-4842-6427-0. – Текст : непосредственный.

43. Weitz, E. Common Lisp Recipes: A Problem-Solution Approach / E. Weitz. – New York : Apress, 2015. – 771 с. – ISBN 978-1-4842-1177-9. – Текст : непосредственный.

44. Richards, M. Fundamentals of Software Architecture A Modern Engineering Approach / M. Richards, N. Ford. – Sebastopol : O'Reilly Media, 2025. – 543 с. – ISBN 978-1-098-17551-1. – Текст : непосредственный.

45. Martin, R. C. Clean Architecture A Craftsman's Guide to Software Structure and Design / R. C. Martin. – Upper Saddle River : Prentice Hall, 2018. – 432 с. – ISBN 978-0-13-449416-6. – Текст : непосредственный.

46. Wiegers, K. E. Software Requirements / K. E. Wiegers, J. Beatty. – Redmond : Microsoft Press, 2013. – 672 с. – ISBN 978-0-7356-7966-5. – Текст : непосредственный.

47. Семакин, И. Г. Основы алгоритмизации и программирования / И. Г. Семакин, А. П. Шестаков. – Москва : Издательский центр «Академия», 2013. – 144 с. – ISBN 978-5-7695-9445-8. – Текст : непосредственный.

48. Белик, А. Г. Проектирование и архитектура программных систем / А. Г. Белик, В. Н. Цыганенко. – Омск : Издательство ОмГТУ, 2016. – 96 с. – ISBN 978-5-8149-2258-8. – Текст : непосредственный.

49. Herda, M. The Common Lisp Condition System: Beyond Exception Handling with Control Flow Mechanisms / M. Herda. – New York : Apress, 2020. – 320 с. – ISBN 978-1-4842-6133-0. – Текст : непосредственный.

50. Knott, G. D. Interpreting LISP: Programming and Data Structures / G. D. Knott. – New York : Apress, 2017. – 163 с. – ISBN 978-1-4842-2706-0. – Текст : непосредственный.

51. Pereira, E. A. The Lisp Handbook: From Syntax to Metaprogramming with Macros / E. A. Pereira. – [б. м.] : [б. и.], 2025. – 168 с. – ISBN 979-8-2981-5582-3. – Текст : непосредственный.

52. Pereira, E. A. Mastering Common Lisp: Advanced Techniques for AI and Web Development / E. A. Pereira. – Seattle : Amazon Digital Services LLC - Kdp, 2025. – 119 с. – ISBN 979-8-2983-7106-3. – Текст : непосредственный.

53. Jones, A. Advanced Techniques in Common LISP: Expert Insights and In-Depth Applications / A. Jones. – [б. м.] : [б. и.], 2024. – 306 с. – ISBN 979-8-3456-7501-4. – Текст : непосредственный.

54. Shangguan, Z. MODERN LISP SYSTEMS PROGRAMMING / Z. Shangguan. – [б. м.] : [б. и.], 2025. – 248 с. – ISBN 979-8-2619-2431-9. – Текст : непосредственный.

55. de Groot, C. The Genius of Lisp / C. de Groot. – Ontario : Berksoft Publications, 2026. – 289 с. – ISBN 978-1-0698-8641-5. – Текст : непосредственный.

56. Prakash, A. A. Compiler Design: Principles and Techniques / A. A. Prakash. – Saarbrücken : Scholars' Press, 2024. – 116 с. – ISBN 978-3-639-70104-3. – Текст : непосредственный.

ПРИЛОЖЕНИЕ А

Представление графического материала

Графический материал, выполненный на отдельных листах, изображен на рисунках А.1–А.11.

Рисунок А.1 – Сведения о ВКРБ

Цель и задачи разработки

Цель настоящей работы – создание оптимизирующего компилятора Common Lisp.

В соответствии с целью работы были сформулированы следующие задачи:

- изучение процесса компиляции языков программирования и функциональных языков в частности;
- изучение методов оптимизации компиляторов;
- проектирование компилятора с оптимизациями;
- реализация компиляции S-выражения в промежуточную форму;
- реализация генерации кода ассемблера;
- реализация генерации байт-кода;
- реализация оптимизации промежуточной формы.

ВКРБ 2206118.09.03.04.26.024			
Имя работы	Фамилия И. О.	Имя	Фамилия
Учебный	Иванов А. С.		
Курсовый	Петров А. А.		
Цель и задачи разработки			
Алгоритм оптимизации			
Выпускная квалификационная работа бакалавра			
03.04.10-216			

Рисунок А.2 – Цель и задачи разработки

Рисунок А.3 – Язык Common Lisp

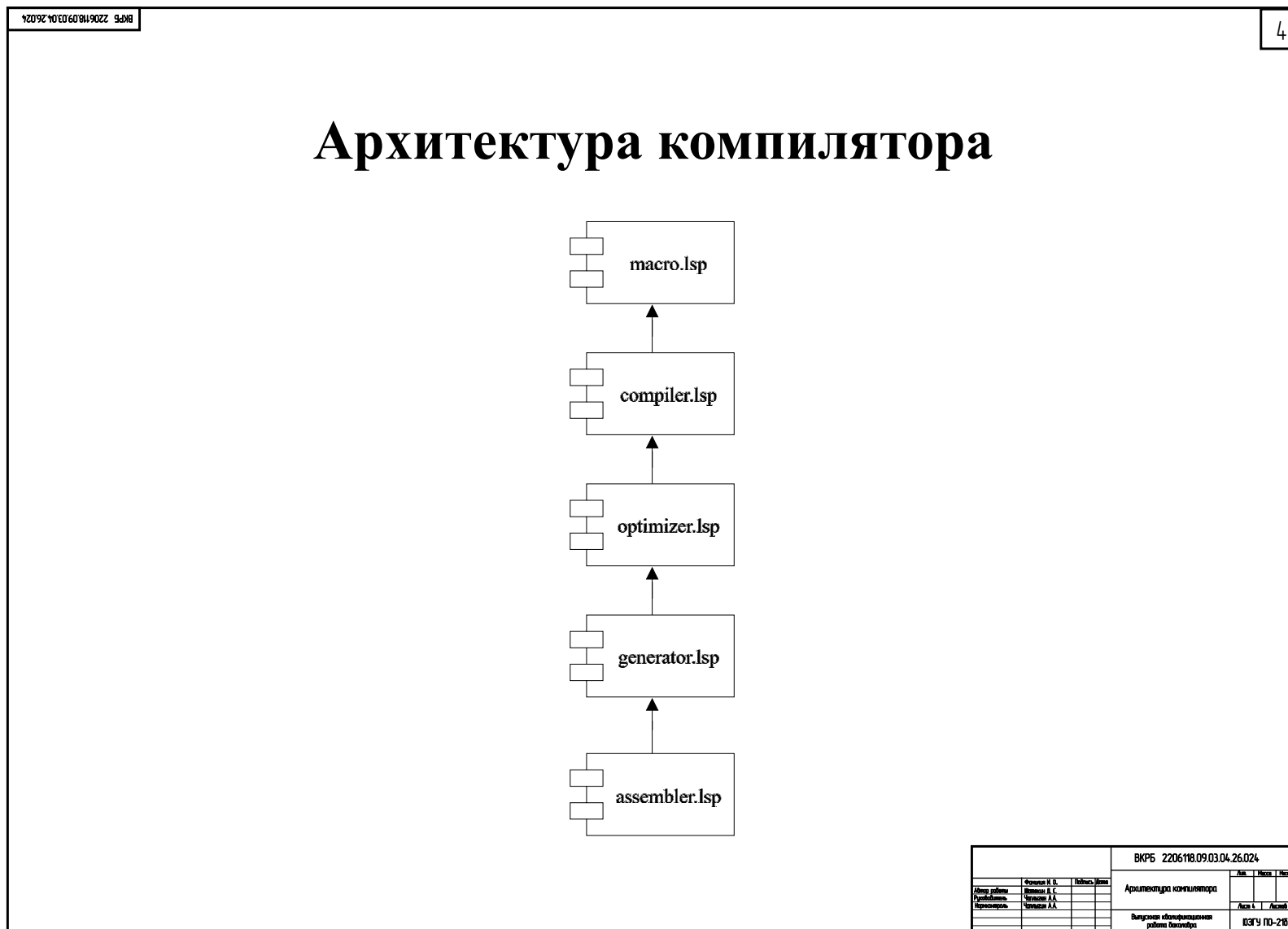


Рисунок А.4 – Архитектура компилятора

Фаза компиляции

Фаза компиляции преобразует исходное S-выражение в промежуточное представление в виде дерева простых операций. Например:

Исходное S-выражение:

```
(defun null (x)
  "Проверка на пустое значение"
  (eq x ()))

(defun caar(x) (car (car x)))
(defun cadar(x) (car (cdr (car x))))

(defmacro let (vars &rest body)
  "Блок локальных переменных"
  `(let ((x ())
        (y 0))
    (+ x y))
    "((lambda (x y)
      (+ x y)) 0 0)"
    `((lambda ,(get-vars vars) ,@body)
      ,(get-vals vars)))

(defun get-vars (v)
  "Получение списка переменных для let"
  (if (null v) nil
      (cons (caar v) (get-vars (cdr v)))))

(defun get-vals (v)
  "Получение списка значений для let"
  (if (null v) nil
      (cons (cadar v) (get-vals (cdr v)))))

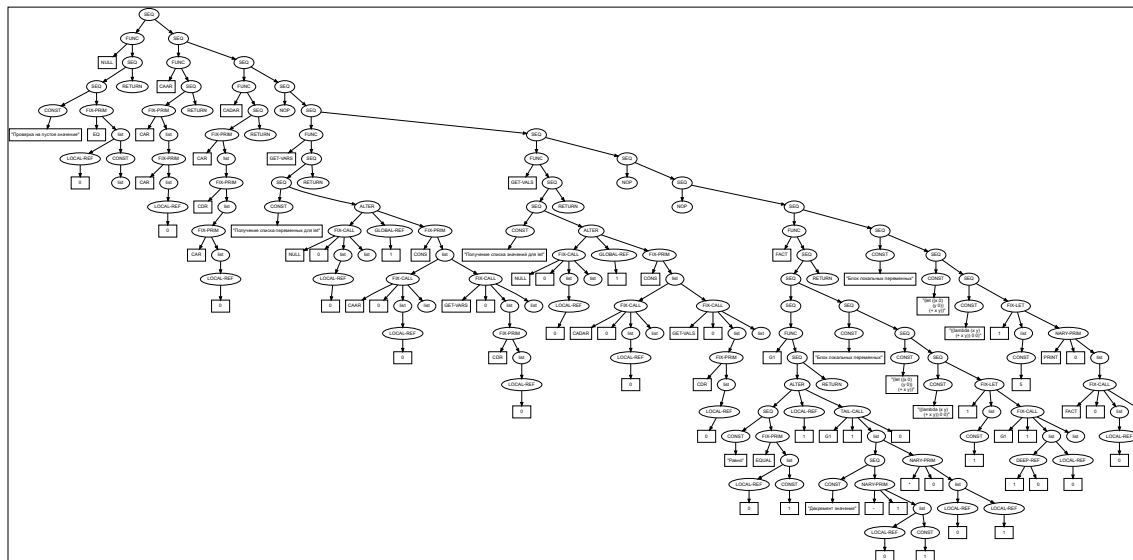
(defmacro = (g1 g2)
  "равно"
  `(equal ,g1 ,g2))

(defmacro -- (x)
  "Декремент значения"
  `(- ,x 1))

(defun fact (n)
  (labels ((fact* (n acc)
            (if (= n 1)
                acc
                (let ((start 1))
                  (fact* (-- n) (* n acc))))))
    (fact* n start)))

(let ((num 5))
  (print (fact num)))
```

Промежуточное дерево:



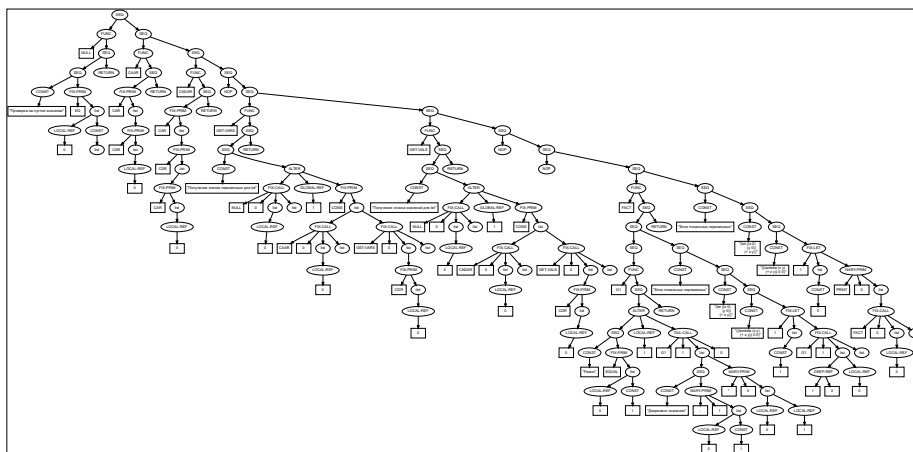
ВКРБ 2206118.09.03.04.26.024			
Автор работы	Филипп И. С.	Надпись	Дата
Рецензент	Филипп А. С.	Фаза компиляции	Акт 5
Проверка	Филипп А. А.	Выполнен оптимизационный	Акт 5
		работы	Акт 5
		Получен	03.04.2016

Рисунок А.5 – Фаза компиляции

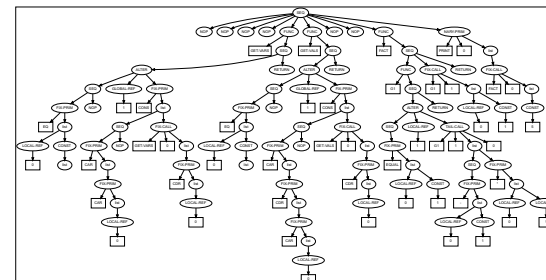
Фаза оптимизации

Фаза оптимизации упрощает промежуточное представление, вычисляет все подвыражения, которые возможно посчитать заранее, подставляет функции, используемые один раз и выполняет различные операции над деревом, чтобы увеличить производительность программы, не меняя её логику. Например:

Дерево перед оптимизацией:



Дерево после оптимизации:



ВКРБ 2206118.09.03.04.26.024			
Имя работы	Фамилия И.О.	Имя	Фамилия
Функционал	Имя И.О.	Имя	Фамилия
Курсовая	Имя И.О.	Имя	Фамилия
Фаза оптимизации		Лит	Место
Выполнен оптимизационный		Лит	Место
работы		03.04.2016	

Рисунок А.6 – Фаза оптимизации

Фаза ассемблирования

Фаза ассемблирования преобразует список линейных инструкций ассемблера в байт-код – массив чисел, включающий в себя опкоды инструкций и их операнды. Байт-код сохраняется в файл в специальном формате. Формат файла должен включать следующие элементы, разделённые пробелами:

- размер байт-кода N (суммарное количество опкодов и операндов);
- непосредственно байт-код размером N (N чисел);
- массив констант (например, \#(T () 0 1 . . .));
- размер массива глобальных переменных;
- хеш-таблица меток и их адресов (например, ((ABS . 2) (G314 . 22) (NOT . 27) . . .)).

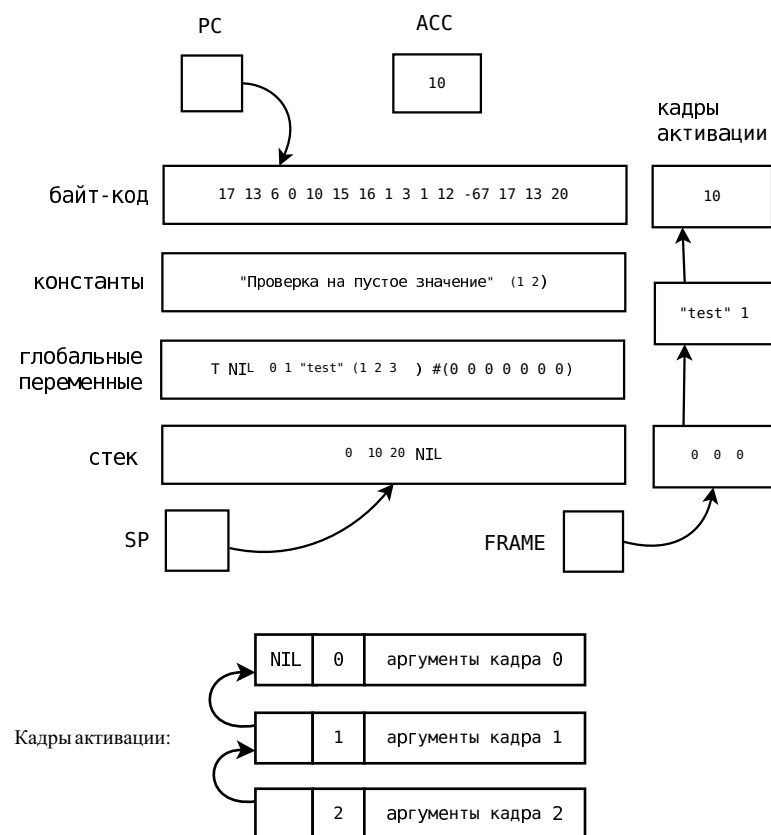
Например, файл, полученный ассемблированием кода из предыдущего плаката:

```
166 1 43 6 0 10 0 1 10 18 9 2 6 4 1 1 28 6 0 10 18 0 10 18 0 10 6 0 10 18 1 10 15 16 0 3 1 12 -34 17 10 18 3 13 1 46 6 0 10 0 1 10 18 9 2 6 4 1 1 31 6
0 10 18 0 10 18 1 10 18 0 10 6 0 10 18 1 10 15 16 0 3 1 12 -37 17 10 18 3 13 1 60 1 43 6 0 10 0 2 10 18 10 2 6 6 1 1 28 6 0 10 0 2 10 18 41 10 6 0 10 6 1
10 18 42 10 25 7 1 25 7 0 1 -38 13 6 0 10 0 2 10 15 16 1 3 2 12 -52 17 13 0 3 10 15 16 0 3 1 12 -66 17 10 11 1 19 9 20 #( T ( ) 1 5 ) 2 ((GET-VARS . 2)
(G5 . 16) (G6 . 42) (G4 . 43) (GET-VALS . 45) (G8 . 59) (G9 . 88) (G7 . 89) (FACT . 91) (G1 . 93) (G12 . 107) (G13 . 133) (G11 . 134) (G10 .
149))
```

ВКРБ 2206118.09.03.04.26.024			
Автор работы	Филипп И. С.	Подпись	Дата
Рецензент	Филипп А. С.		
Корректор	Филипп А. А.		
Фаза ассемблирования		Лит	Начи
Выпускная квалификационная работа		Лит	Лит
Итого		03/04 10-216	

Рисунок А.8 – Фаза ассемблирования

Архитектура виртуальной машины



			ВКРБ 2206118.09.03.04.26.024				
			Архитектура ВМ		Авт.	Изд.	Исп.
Автор работы	Фролина И. В.	Подпись			Лист 9		Листов 11
Руководитель	Васильев А. С.						
Нормировщик	Чепелев А.А.						
	Чепелев А.А.						
			Выпускная квалификационная работа бакалавра				03/19 ПО-216

112

Рисунок А.10 – Тестирование

Заключение

Разработанная программная система оптимизирующего компилятора Common Lisp представляет собой полностью функциональный транслятор исходного кода на языке Common Lisp в байт-код, реализующий все основные возможности языка, а также виртуальную машину для выполнения исходных программ.

Основные результаты работы:

1. Изучен процесс компиляции языков программирования и функциональных языков в частности.
2. Изучены методы оптимизации компиляторов.
3. Спроектирован компилятор с оптимизациями.
4. Реализована компиляция S-выражения в промежуточную форму.
5. Реализована генерация кода ассемблера.
6. Реализована генерация байт-кода.
7. Реализована оптимизация промежуточной формы.

Все требования, объявленные в техническом задании, были полностью реализованы, все задачи, поставленные в начале разработки проекта, были также решены.

ВКРБ 2206118.09.03.04.26.024			
Имя	Фамилия	Имя	Фамилия
Имя	Фамилия	Имя	Фамилия
Имя	Фамилия	Имя	Фамилия
Заключение			
Выпускник специализации			
работы по теме			
03.04.2026			

Рисунок А.11 – Заключение

ПРИЛОЖЕНИЕ Б

Фрагменты исходного кода программы

compiler.lsp

```
1 ;; *global-variables* - список имён глобальных переменных.
2 (defvar *global-variables*)
3 ;; *global-variables-count* - число глобальных переменных в списке *
  global-variables*.
4 (defvar *global-variables-count*)
5 ;; *fix-functions* - глобальное окружение функций, содержащий названия
  функций, смещение кадра окружения, кол-во аргументов.
6 (defvar *fix-functions*)
7 ;; список функций с переменным числом аргументов
8 (defvar *nary-functions*)
9 ;; список локальных функций (локальное имя, смещение кадра, кол-во аргументов
  , скомпилированное тело)
10 (defvar *local-functions*)
11 ;; есть ли comma-at в коде
12 (defvar *comma-at*)
13 ;; текущая функция для проверки рекурсии
14 (defvar *current-func*)
15 ;; хэш скомпилированных значений констант по имени переменной
16 (defvar *const-vals*)
17 ;; хэш информации о функциях, ключ - имя,
18 ;; внутри хеш {count, rec, body, can-inline, closure}
19 ;; число обращений, рекурсивная?, оптимизированное тело, встраиваемая?, есть
  замыкание на нее?
20 (defvar *functions-info*)
21
22 (defun extend-env (env args)
23   ";; Расширить окружение новым кадром аргументов"
24   (cons args env))
25
26 (mk/add-func add-func *fix-functions* args body) ;; фиксированное число
  аргументов
27 (mk/add-func add-nary-func *nary-functions* args body) ;; переменное число
  аргументов
28 (mk/add-func add-local-func *local-functions* real-name) ;; локальные функции
29
30 (defun inner-compile (expr env tail) nil)
31
32 (defun compile-func-body (name args body env)
33   ";; Компиляция тела функции"
34   (when (not (check-key *functions-info* name))
35     (set-hash *functions-info* name (make-hash))
36     (set-hash (get-hash *functions-info* name) 'count 0))
37   (list 'FUNC name
38     (list 'SEQ
39       (inner-compile body (extend-env env args) t)
40       (list 'RETURN))))
41
42 (defun compile-lambda (name args body env)
43   ";; Компирует лямбда-абстракцию."
```

```

44 ;; (список аргументов, тело функции, локальное окружение).
45 (if (is-nary args)
46     (add-nary-func name (list-length env) (num-fix-args args 0) args body)
47     (add-func name (list-length env) (list-length args) args body))
48 (compile-func-body name args body env)
49
50 (defun find-func (f)
51     ;; Определения типа функции: лямбда, примитив, nary примитив, глобальная
    функция, nary функция"
52     (if (and (not (atom f)) (correct-lambda f))
53         (list 'lambda (list-length (second f)) (gensym)) ; lambda num-args name
54         (let ((r nil))
55             (cond
56                 ((setq r (search-symbol *fix-macros* f))
57                  (list 'fix-macro (third r) (forth r) (fifth r))) ; fix-macro num-args
                    args body
58                 ((setq r (search-symbol *nary-macros* f))
59                  (list 'nary-macro (third r) (forth r) (fifth r))) ; nary-macro num-args
                    args body
60                 ((setq r (search-symbol *local-functions* f))
61                  (list 'local-func (third r) (second r) (forth r))) ; local-func
                    num-args env real-name
62                 ((setq r (search-symbol *fix-functions* f))
63                  (list 'fix-func (third r) (second r) (forth r) (fifth r))) ; fix-func
                    num-args env args body
64                 ((setq r (search-symbol *nary-functions* f))
65                  (list 'nary-func (third r) (second r) (forth r) (fifth r))) ; nary-func
                    num-args env args body
66                 ((setq r (search-symbol *nary-primitives* f))
67                  (list 'nary-prim (cdr r))) ; nary-prim num-args
68                 ((setq r (search-symbol *fix-primitives* f))
69                  (list 'fix-prim (cdr r))) ; fix-prim num-args
70                 (t (comp-err "unknown function " f))))))
71
72 (defun compile-progn (lst env tail) nil)
73
74 (defun inc-function-ref (name)
75     "Увеличить число ссылок на функцию"
76     (if (check-key *functions-info* name)
77         (let* ((f (get-hash *functions-info* name))
78                (count (get-hash f 'count)))
79             (set-hash f 'count (+ count)))
80         (progn (set-hash *functions-info* name (make-hash))
81                (set-hash (get-hash *functions-info* name) 'count 1))))
82
83 (defun compile-application (f args env tail)
84     ;; Применение функции"
85     ;; f - имя функции или lambda, args - аргументы, env - окружение
86     (let* ((fun (find-func f))
87            (type (car fun))
88            (count (second fun))
89            (cur-env (list-length env))
90            (rec (eq f *current-func*))
91            (is-tail-call (and tail rec)))

```

```

92 (name (case type ('lambda (third fun)) ('local-func (forth fun)) (
    otherwise f))))
93 (check-arguments f type count args)
94 (when (contains '(fix-func nary-func local-func lambda) type)
95   (inc-function-ref name))
96 (when rec (set-hash (get-hash *functions-info* name) 'rec t))
97 (if (eq type 'fix-macro)
98   (inner-compile (macroexpand (third fun) args (forth fun)) env tail)
99   (if (eq type 'nary-macro)
100     (let ((r (macroexpand (third fun) (make-nary-args count args) (forth
        fun))))
101       (inner-compile r env tail))
102     (let ((vals (map #'(lambda (a) (inner-compile a env nil)) args))) ;;
        параметры - не хвостовой вызов
103     (case type
104       ('lambda (list 'FIX-LET count vals (compile-progn (cddr f) (extend-env
        env (second f)) tail)))
105       ('local-func (list (if is-tail-call 'TAIL-CALL 'FIX-CALL)
106                           (forth fun) (third fun) vals (when is-tail-call (-
        cur-env (++ (third fun))))))
107       ('fix-func (list (if is-tail-call 'TAIL-CALL 'FIX-CALL)
108                         f (third fun) vals (when is-tail-call (- cur-env (++
        (third fun))))))
109       ('nary-func (list (if is-tail-call 'TAIL-NCALL 'NARY-CALL)
110                         f count (third fun) vals (when is-tail-call (-
        cur-env (++ (third fun))))))
111       ('fix-prim (list 'FIX-PRIM f vals))
112       ('nary-prim (list 'NARY-PRIM f count vals))))))
113
114 (defun compile-function (f env)
115   ";; Создание функции-замыкания"
116   (let* ((fun (find-func f))
117          (type (car fun))
118          (name (gensym)))
119     (when (contains '(fix-func nary-func local-func lambda) type)
120       (inc-function-ref (case type ('lambda name) ('local-func (forth fun)) (
        otherwise f))))
121     (case type
122       ('lambda (list 'FIX-CLOSURE name (list-length env)
123         (let ((old-func *current-func*)
124              (res nil))
125           (setq *current-func* nil
126                res (compile-lambda name (second f) (cons 'progn (cddr f)) env)
127                *current-func* old-func)
128           res)))
129       ('fix-func (list 'FIX-CLOSURE f (third fun) nil))
130       ('local-func (list 'FIX-CLOSURE (forth fun) (third fun) nil))
131       ('fix-prim (list 'PRIM-CLOSURE f))
132       ('nary-prim (list 'NPRIM-CLOSURE f))
133       (otherwise (comp-err "invalid function argument" f fun))))
134
135 (defun compile-defun (expr env)
136   ";; Компилирует объявление функции с помощью DEFUN."
137   ;; expr - список, состоящий из названия, списка аргументов и тела функции.

```

```

138 ;; проверка на правильность expr
139 (let ((name (car expr))
140       (args (second expr))
141       (body (cddr expr))
142       (old-func *current-func*)
143       (res nil))
144     (setq *current-func* name
145           res (compile-lambda name args (cons 'progn body) env)
146           *current-func* old-func)
147     res))
148
149 (defun add-global (sym)
150   ";; Добавить глобальную переменную"
151   ;; Возвращает индекс добавленной переменной
152   (setq *global-variables* (append *global-variables* (list sym)))
153   (incf *global-variables-count*)
154   (- *global-variables-count* 1))
155
156 (defun find-global-var (var)
157   ";; Производит поиск переменной в глобальном окружении по символу."
158   ;; Возвращает индекс в массиве глобального окружения, если символ найден,
159   ;; иначе nil.
160   (let ((i (list-search *global-variables* var)))
161     (if (null i) nil
162         (list 'global i))))
163
164 (defun var-search(params var)
165   ";; Вычисление порядкового номера переменной в списке с учетом &rest"
166   ;; Если не найдено - возвращает nil
167   (labels ((find (list pos)
168             (cond
169              ((null list) nil)
170              ((eq (car list) '&rest) (find (cdr list) pos))
171              ((eq (car list) var) pos)
172              (t (find (cdr list) (++ pos))))))
173     (find params 0)))
174
175 (defun find-local-var (var env)
176   ";; Производит поиск переменной в локальном окружении по символу."
177   ;; Возвращает индекс переменной в стеке, если символ найден, иначе nil.
178   (labels ((find-frame (frames i)
179             (if (null frames) nil
180                 (let ((j (var-search (car frames) var)))
181                   (if (null j) (find-frame (cdr frames) (++ i))
182                       (if (= i 0) (list 'local j)
183                           (list 'deep i j)))))))
184     (find-frame env 0)))
185
186 (defun find-var (var env)
187   ";; Производит поиск переменной по символу сначала в локальном, а затем в
188   ;; глобальном окружении."
189   ;; Возвращает список, состоящий из символа - GLOBAL или LOCAL, DEEP,
190   ;; env - локальное окружение
191   (let ((local (find-local-var var env)))

```

```

190     (if (not (null local)) local
191         (find-global-var var))))
192
193 (defun compile-setq (body env)
194     ";; Компилирует setq-выражение."
195     ;; setq-body - пара из символа и выражения, которое необходимо установить
196     ;; этому символу.
197     (if (null body)
198         (comp-err "setq: no params")
199         (labels ((compile-all-setq (body)
200                     (if (= (list-length body) 2)
201                         (compile-single-setq body)
202                         (list 'SEQ (compile-single-setq body) (compile-all-setq
203                             (cddr body))))))
204             (compile-single-setq (body)
205                 (if (< (list-length body) 2)
206                     (comp-err "setq: no expression to set")
207                     (let* ((var (car body))
208                            (val (inner-compile (second body) env nil))
209                            (res (find-var var env)))
210                         (when (not (symbolp var))
211                             (comp-err "setq: invalid variable: " var))
212                         (when (check-key *const-vals* var)
213                             (comp-err "setq: can't modify a constant variable:"
214                                 var))
215                         (if (null res)
216                             (list 'GLOBAL-SET (add-global var) val)
217                             (case (car res)
218                                 ('global (list 'GLOBAL-SET (second res) val))
219                                 ('local (list 'LOCAL-SET (second res) val))
220                                 ('deep (list 'DEEP-SET (second res) (third res)
221                                     val))
222                                 (otherwise (error "Unreachable"))))))))
223             (compile-all-setq body))))
224
225 (defun compile-if (expr env tail)
226     ";; Компилирует if-выражение."
227     ;; if-body - тело if-выражения (условие, ветка по "Да" и по "Нет").
228     (if (not (= (list-length expr) 3))
229         (comp-err "if: invalid params")
230         (let ((cond (inner-compile (car expr) env nil))
231               (true (inner-compile (second expr) env tail))
232               (false (inner-compile (third expr) env tail)))
233             (list 'ALTER cond true false))))
234
235 (defun compile-constant (c)
236     ";; Компиляция константы"
237     ;; (print (list 'compile-constant c))
238     (list 'CONST c))
239
240 (defun compile-progn (lst env tail)
241     ";; Компилирует блок progn."
242     ;; lst - список S-выражений внутри блока progn.
243     (if (null lst) (compile-constant '())
244         (let ((res (inner-compile (car lst) env nil)))
245             (if (null res) (compile-progn (cdr lst) env tail)
246                 (list 'ALTER res (compile-progn (cdr lst) env tail))))))

```

```

240     (if (null (cdr lst))
241         (inner-compile (car lst) env tail)
242         (list 'SEQ (inner-compile (car lst) env nil) (compile-progn (cdr lst)
                               env tail))))))
243
244 (defun compile-labels (lst env tail)
245     ";; Компилирует labels."
246     ;; lst - (<список функций> <форма1> ... <форман>).
247     (let ((old *local-functions*)
248           (old-func *current-func*))
249         (labels ((extend-func (funcs)
250                     (app
251                      #'(lambda (f)
252                          (add-local-func (car f) (list-length env) (list-length (second f)) (gensym))) funcs))
253                   (compile-funcs (funcs)
254                                   (cons 'SEQ (map
255                                           #'(lambda (f)
256                                               (let ((name (forth (find-func (car f))))
257                                                   (args (second f))
258                                                   (body (cddr f)))
259                                                   (setf *current-func* (car f))
260                                                   (compile-func-body name args (cons 'progn body) env)))
261                                           funcs))))
262                   (extend-func (car lst))
263                   (let ((f (compile-funcs (car lst)))
264                         (body (progn (setf *current-func* old-func) (compile-progn (cdr lst)
265                                             env tail))))
265                       (setf *local-functions* old)
266                       (list 'SEQ f body))))))
267
268 (defun compile-variable (v env)
269     ";; Компиляция переменной"
270     (if (check-key *const-vals* v)
271         (get-hash *const-vals* v)
272         (let ((res (find-var v env)))
273             (if (null res)
274                 (comp-err "Unknown symbol" v)
275                 (case (car res)
276                     ('local (list 'LOCAL-REF (second res)))
277                     ('global (list 'GLOBAL-REF (second res)))
278                     ('deep (list 'DEEP-REF (second res) (third res)))))))
279
280 (defun compile-backquote (expr env)
281     ";; Компиляция квазицитирования"
282     ;; (print (list 'compile-backquote expr))
283     ;; (when (null expr)
284     ;;     (comp-err "backquote: no body"))
285     (cond ((atom expr) (compile-constant expr))
286           ((equal (car expr) 'COMMA) (inner-compile (second expr) env nil))
287           ((and (pairp (car expr)) (not (null (car expr))) (equal (caar expr) '
288                               COMMA-AT)) (progn (setf *comma-at* t)
289                                                    (list 'FIX-CALL 'append2 0 (list (inner-compile (cadar expr) env nil) (
290                                                                 compile-backquote (cdr expr) env))))))

```

```

289 (t (list 'FIX-PRIM 'CONS (list (compile-backquote (car expr) env) (
    compile-backquote (cdr expr) env))))))
290
291 (defun compile-tagbody (body env)
292   ";; Компиляция tagbody"
293   (if (null body) (list 'CONST 'nil)
294       (list 'SEQ (let ((e (car body)))
295                     (if (symbolp e) (list 'LABEL e)
296                         (inner-compile e env nil)))
297         (compile-tagbody (cdr body) env))))
298
299 (defun compile-catch (args env)
300   ";; Компиляция CATCH"
301   (list 'CATCH (inner-compile (car args) env nil) (compile-progn (cdr args)
    env nil)))
302
303 (defun compile-throw (args env)
304   ";; Компиляция THROW"
305   (list 'THROW (inner-compile (car args) env nil) (inner-compile (second
    args) env nil)))
306
307 (defun compile-defconst (body env tail)
308   "Компиляция DEFCONST, запомним константу"
309   (let ((var (car body))
310         (expr (if (null (cdr body)) nil (second body))))
311     (when (check-key *const-vals* var)
312       (comp-err "defconst: redefinition of const:" var))
313     (set-hash *const-vals* var (inner-compile expr env tail))
314     (list 'NOP)))
315
316 (defun compile-apply (func args pack env tail)
317   "Компиляция применения функции к аргументам"
318   (list 'APPLY (inner-compile func env tail) args pack))
319
320 (defun compile-args (args env)
321   "Компиляция аргументов для APPLY"
322   (map #'(lambda (a) (inner-compile a env nil)) args))
323
324 (defun inner-compile (expr env tail)
325   "Рекурсивная функция компиляции в промежуточную форму"
326   ;; expr - выражение, env - лексическое окружение, tail - если t, то
    выражение в хвостовой позиции, иначе nil
327   ;; (print (list 'inner-compile expr))
328   (if (atom expr)
329       (if (symbolp expr)
330           (compile-variable expr env)
331           (compile-constant expr))
332       (let ((func (car expr))
333             (args (cdr expr)))
334         (case func
335           ('quote (compile-constant (second expr)))
336           ('progn (compile-progn args env tail))
337           ('if (compile-if args env tail))
338           ('setq (compile-setq args env))

```



```

339 ('defun (compile-defun args env))
340 ('labels (compile-labels args env tail))
341 ('function (compile-function (second expr) env))
342 ('defmacro (progn (make-macro args) (list 'NOP)))
343 ('backquote (compile-backquote (second expr) env))
344 ('tagbody (compile-tagbody (cdr expr) env))
345 ('go (list 'GOTO (second expr)))
346 ('catch (compile-catch (cdr expr) env))
347 ('throw (compile-throw (cdr expr) env))
348 ('defconst (compile-defconst (cdr expr) env tail))
349 ('funcall (compile-apply (car args) (compile-args (cdr args) env) t env
tail))
350 ('apply (if (search-symbol *local-functions* 'apply) (compile-application
func args env tail)
351 (compile-apply (car args) (list (inner-compile (second args) env
nil)) nil env tail)))
352 (otherwise (compile-application func args env tail))))))
353
354 (defun compile (expr)
355 "Компиляция в промежуточную форму"
356 (setq *global-variables* '(t nil)
357 *global-variables-count* (list-length *global-variables*)
358 *comma-at* nil
359 *comp-err* nil
360 *comp-err-msg* nil
361 *fix-functions* nil
362 *environment* nil
363 *const-vals* (make-hash)
364 *functions-info* (make-hash))
365 (let ((c (inner-compile expr nil t))) ;; начальный вызов - хвостовой
366 (if *comma-at* (list 'SEQ (inner-compile
367 '(defun append2 (l1 l2) (if (eq l1 nil) l2 (cons (car l1) (
append2 (cdr l1) l2))))
nil nil) c) c)))
368

```

optimizer.lsp

```

1 (defvar *optimize-flags* ; список включенных флагов оптимизации
промежуточного дерева
2 '(trivial-condition
3 simplify-arithmetic
4 dead-code-elimination
5 tail-call
6 constant-folding
7 unused-functions
8 beta-expansion
9 all-inline))
10
11 (defconst +const-fix-primst+ ; список FIX-примитивов, которые можно посчитать
заранее на этапе компиляции
12 '(car cdr atom cons
13 %<< >> eq equal > < sin cos sqrt
14 intern symbol-name string-size inttostr code-char char-code char subseq
15 array-size aref symbolp integerp pairp functionp
16 round ~ + - * / & bitor ^ arrayp stringp))

```

```

17 (defconst +const-nary-prim+ ; список NARY-примитивов, которые можно
    посчитать заранее на этапе компиляции
18 '(+ - * / & bitor ^ concat))
19
20 (defun accumulator-loading (tree)
21   "Удаляет избыточные формы загрузки значений в аккумулятор"
22   (labels ((is-same-param (set ref)
23             (let ((set-expr (if (eq (car set) 'DEEP-SET) (forth set) (third
24                                   set)))))
25             (and (contains '(CONST GLOBAL-REF LOCAL-REF DEEP-REF
26                             FIX-CLOSURE PRIM-CLOSURE NPRIM-CLOSURE) (car set-expr))
27                  (equal set-expr ref)))))
26   (is-set-ref (set ref)
27     (and (not (null set))
28          (contains '(GLOBAL-SET LOCAL-SET DEEP-SET) (car set))
29          (or (and (or (and (eq (car set) 'GLOBAL-SET) (eq (car ref)
30                                                             'GLOBAL-REF))
31                     (and (eq (car set) 'LOCAL-SET) (eq (car ref) '
32                                                             LOCAL-REF))
33                     (and (eq (car set) 'DEEP-SET) (eq (car ref) '
34                                                             DEEP-REF))
35                     (eq (second set) (second ref))))
36                (is-same-param set ref)))))
34   (remove-set-ref (prev-elem subtree)
35     (if (null subtree)
36         nil
37         (let* ((cur-elem (car subtree))
38                (next-elem (remove-set-ref cur-elem (cdr subtree))))
39           (if (is-set-ref prev-elem cur-elem)
40               next-elem
41               (cons cur-elem next-elem))))))
42   (cons 'SEQ (remove-set-ref nil (cdr tree)))))
43
44 (defun trivial-condition (alter)
45   "Если условие формы ALTER - константа, то считает значение условия и
46    сокращает форму ALTER до одной из её веток"
47   (if (contains *optimize-flags* 'trivial-condition)
48       (let ((cond (second alter))
49             (true (third alter))
50             (false (forth alter)))
51         (if (or (eq (car cond) 'CONST)
52                 (and (eq (car cond) 'GLOBAL-REF)
53                      (contains '(0 1) (cadr cond))))
54             (let ((cond-val (case (car cond)
55                                ('CONST (cadr cond))
56                                ('GLOBAL-REF (case (cadr cond) (0 t) (1 nil) (
57                                                        otherwise nil))))
58                 (otherwise nil))))
59               (if cond-val true false))
60         alter)) alter))
61
62 (defun simplify-arithmetic (tree)

```

```

61 "Заменяет вызов арифметического примитива tree с неограниченным кол-вом
    аргументов на вложенные вызовы примитивов с фиксированным кол-вом
    аргументов"
62 (if (contains *optimize-flags* 'simplify-arithmetic)
63     (let ((prim (second tree))
64           (args (forth tree)))
65         (if (and (contains '(+ - * / & bitor ^) prim)
66             (>= (list-length args) 2))
67             (foldl #'(lambda (prev cur)
68                       `(FIX-PRIM ,prim (,prev ,cur)))
69                 (car args) (cdr args))
70             tree)) tree))
71
72 (defun constant-folding (tree)
73     "Свёртка и распространение констант"
74     (labels ((try-compute (tree)
75         "Пытается вычислить значение промежуточной формы, если она
76         константа"
77         "Возвращает список, состоящий из одного элемента - вычисленного
78         выражения, либо nil"
79         (case (car tree)
80             ('CONST (list (second tree)))
81             ('GLOBAL-REF (case (second tree) (0 (list t)) (1 (list nil)) (
82                 otherwise nil))))
83             (otherwise
84                 (when (or (and (eq (car tree) 'FIX-PRIM) (contains +
85                     const-fix-prim+ (second tree)))
86                     (and (eq (car tree) 'NARY-PRIM) (contains +
87                         const-nary-prim+ (second tree))))
88                     (let ((prim (second tree))
89                           (vals (map #'try-compute (if (eq (car tree) 'FIX-PRIM
90                               ) (third tree) (forth tree)))))
91                         (unless (contains vals nil)
92                             (list (apply (symbol-function prim) (map #'car vals))))
93                         )))))
94         (if (contains *optimize-flags* 'constant-folding)
95             (let ((const-val (try-compute tree))
96                   (if (null const-val)
97                       tree
98                       (list 'CONST (car const-val))))
99                 tree)))
100
101 (defun expand-seqs (tree)
102     "Разворачивает вложенные SEQ"
103     "tree - SEQ-форма"
104     (labels ((expand (subtree)
105         (if (null subtree)
106             nil
107             (let ((cur-elem (car subtree))
108                   (next-elems (expand (cdr subtree))))
109                 (if (eq (car cur-elem) 'SEQ)
110                     (append (expand (cdr cur-elem)) next-elems)
111                     (cons cur-elem next-elems))))))
112         (cons 'SEQ (expand (cdr tree)))))

```

```

106
107 (defun dead-code-elimination (tree)
108   "Убирает из формы SEQ неиспользуемый код"
109   "К неиспользованному коду относится:"
110   "- последовательность команд загрузки аккумулятора (кроме последней
      команды)"
111   (labels ((remove-acc-loading (subtree)
112             (let ((cur-elem (car subtree))
113                   (rest-elems (cdr subtree)))
114               (cond ((null rest-elems) (list cur-elem))
115                     ((and (not (eq (car (second subtree)) 'RETURN))
116                           (contains '(CONST GLOBAL-REF LOCAL-REF DEEP-REF) (
117                                     car cur-elem)))
118                      (remove-acc-loading rest-elems))
119                     (t (cons cur-elem (remove-acc-loading rest-elems))))))
119   (if (contains *optimize-flags* 'dead-code-elimination)
120       (accumulator-loading (cons 'SEQ (remove-acc-loading (cdr (expand-seqs
121       tree))))
122       tree)))
123 (defun tail-call (tree)
124   "Оптимизация хвостовой рекурсии"
125   "tree - форма TAIL-CALL или TAIL-NCALL"
126   (if (contains *optimize-flags* 'tail-call)
127       tree
128       (cons (if (eq (car tree) 'TAIL-CALL) 'FIX-CALL 'NARY-CALL) (cdr tree)))
129   )
130 (defun unused-functions (tree)
131   "Удаление неиспользуемых функций"
132   "tree - LABEL-форма"
133   (if (contains *optimize-flags* 'unused-functions)
134       (if (and (check-key *functions-info* (second tree))
135               (= (get-hash (get-hash *functions-info* (second tree)) 'count) 0))
136           (list 'NOP) tree)
137       tree))
138
139 (defun search-abs-tree (exp nodes)
140   "Возвращает истину если найдены хотя бы один узел из списка nodes"
141   (cond ((null exp) nil)
142         ((atom exp) nil)
143         ((and (pairp exp) (atom (car exp))) (or (contains nodes (car exp)) (
144           search-abs-tree (cdr exp) nodes)))
145         (t (or (search-abs-tree (car exp) nodes) (search-abs-tree (cdr exp) nodes)
146           )))
147
148 (defun one-ref-args (body)
149   "К каждому аргументу только одно обращение"
150   (let ((counts (make-hash)))
151     (labels ((refs (exp)
152               (cond ((null exp) t)
153                     ((atom exp) t)
154                     ((and (pairp exp) (atom (car exp))) (eq (car exp) 'LOCAL-REF))

```

```

153         (if (check-key counts (second exp)) nil (set-hash counts (second
154             exp) t)))
155     (t (and (refs (car exp)) (refs (cdr exp))))))
156
157 (defun search-closure-deep-ref (exp)
158     "Ищем все fix-closure, и проверяем тело замыкания на deep-ref"
159     (cond ((null exp) t)
160           ((atom exp) t)
161           ((and (pairp exp) (atom (car exp)) (eq (car exp) 'FIX-CLOSURE)) (not (
162               search-abs-tree (forth exp) '(DEEP-REF))))
163           (t (and (search-closure-deep-ref (car exp)) (search-closure-deep-ref (cdr
164               exp))))))
165
166 (defun beta-exp (exp args level)
167     "Выполнить подстановку тела функции"
168     ;;(print `(beta-exp ,exp))
169     (cond ((null exp) nil)
170           ((atom exp) exp)
171           ((and (pairp exp) (atom (car exp)))
172            (case (car exp)
173                  ('FIX-CLOSURE (list 'FIX-CLOSURE (second exp) (third exp) (beta-exp (
174                      forth exp) args (++ level))))
175                  ('FUNC (let* ((f (get-hash *functions-info* (second exp)))
176                                (dc (if (check-key f 'decl-closure) (get-hash f 'decl-closure) 0)))
177                          (set-hash f 'decl-closure (++ dc))
178                          (if (< dc 2) (list 'FUNC (second exp) (beta-exp (third exp) args
179                              level)) (list 'NOP))))
180                  ('FIX-LET (if (null (cdr exp)) exp (list 'FIX-LET (second exp) (third
181                      exp) (beta-exp (forth exp) args (++ level))))
182                  ('LOCAL-REF (if (= level 0) (nth args (second exp)) exp))
183                  ('DEEP-REF (let ((lev (second exp)))
184                              (cond ((= lev level) (nth args (third exp)))
185                                    ((= lev 1) (list 'LOCAL-REF (third exp)))
186                                    ((> lev level) (list 'DEEP-REF (-- lev) (third exp)))
187                                    (t (error "DEEP-REF lev < level")))))
188                  ('RETURN (if (> level 0) exp (list 'NOP)))
189                  (otherwise (cons (beta-exp (car exp) args level) (beta-exp (cdr exp)
190                      args level))))))
191     (t (cons (beta-exp (car exp) args level) (beta-exp (cdr exp) args level))))
192
193 (defun make-nary-param (num args)
194     "Создать выражение для подстановки аргументов args, num - число
195     фиксированных аргументов"
196     (cond ((null args) (cons (list 'GLOBAL-REF 1) ()))
197           ((= num 0) (cons (list 'NARY args) ()))
198           (t (cons (car args) (make-nary-param (-- num) (cdr args))))))
199
200 (defun beta-expansion (call)
201     "Подстановка тела функции, когда функция вызывается один раз и это возможно"
202
203     (let* ((f (get-hash *functions-info* (second call)))
204            (count (get-hash f 'count)))

```

```

197 (body (get-hash f 'body))
198 (can-inline (get-hash f 'can-inline)))
199 (if (and can-inline (or (= count 1) (contains *optimize-flags* '
200 all-inline)))
201 (progn ;;(print `(beta-expansion ,call))
202 (beta-exp body (if (eq (car call) 'FIX-CALL) (forth call)
203 (make-nary-param (third call) (fifth call))) 0) call)))
204 (defun can-inline (f)
205 "Определение возможно ли встраивание функции"
206 (let* ((rec (check-key f 'rec))
207 (body (if rec nil (get-hash f 'body))))
208 (and (not rec)
209 (not (search-abs-tree body '(LOCAL-SET DEEP-SET))) (one-ref-args body)
210 (search-closure-deep-ref body))))
211
212 (defun side-effects (exp)
213 "Есть ли в выражении побочные эффекты"
214 (search-abs-tree exp '(LOCAL-SET GLOBAL-SET DEEP-SET)))
215
216 (defun optimize-tree1 (tree)
217 "Оптимизация промежуточной формы tree методами, указанными флагами flags"
218 "Возвращает оптимизированное дерево"
219 (labels ((optimize-many (tree) (map #'optimize tree))
220 (optimize-cdr (tree) (cons (car tree) (optimize-many (cdr tree))))
221 (optimize-nth (tree n) (if (= n 1)
222 (cons (optimize (car tree)) (cdr tree))
223 (cons (car tree) (optimize-nth (cdr tree) (-- n)))))
224 (optimize (tree)
225 (if (contains '(NOP CONST GLOBAL-REF LOCAL-REF DEEP-REF RETURN
226 PRIM-CLOSURE NPRIM-CLOSURE GOTO LABEL) (car tree))
227 tree
228 (case (car tree)
229 ('SEQ (dead-code-elimination (optimize-cdr tree)))
230 ('ALTER (trivial-condition (optimize-cdr tree)))
231 ('NARY-PRIM (constant-folding (simplify-arithmetic
232 (list (car tree) (second tree) (third tree) (optimize-many (forth
233 tree))))))
234 ('FUNC (let ((body (optimize (third tree))))
235 (when (check-key *functions-info* (second tree))
236 (let* ((f (get-hash *functions-info* (second tree)))
237 (dc (if (check-key f 'decl-count) (get-hash f 'decl-count) 0)))
238 (set-hash f 'body body)
239 (set-hash f 'decl-count (++ dc))
240 (if (check-key f 'can-inline) (set-hash f 'can-inline nil)
241 (set-hash f 'can-inline (can-inline f)))))
242 (unused-functions (list (car tree) (second tree) body))))
243 ('FIX-CLOSURE (if (null (forth tree))
244 (progn (set-hash (get-hash *functions-info* (second tree)) '
245 closure t) tree)
246 (optimize-nth tree 4)))
247 ('GLOBAL-SET (optimize-nth tree 3))
248 ('LOCAL-SET (optimize-nth tree 3))
249 ('DEEP-SET (optimize-nth tree 4))

```

```

247 ('FIX-LET (list (car tree) (second tree) (optimize-many (third tree))
              (optimize (forth tree))))
248 ('FIX-PRIM (constant-folding (list (car tree) (second tree) (
              optimize-many (third tree)))))
249 ('FIX-CALL (let ((args (optimize-many (forth tree))))
250              (when (side-effects args)
251                  (set-hash (get-hash *functions-info* (second tree)) 'can-inline nil
                              ))
              (list (car tree) (second tree) (third tree) args)))
252 ('TAIL-CALL (tail-call (list (car tree) (second tree) (third tree) (
253              optimize-many (forth tree)) (fifth tree)))))
254 ('NARY-CALL (list (car tree) (second tree) (third tree) (forth tree) (
              optimize-many (fifth tree)))))
255 ('TAIL-NCALL (tail-call (list (car tree) (second tree) (third tree) (
              forth tree) (optimize-many (fifth tree)) (sixth tree)))))
256 ('CATCH (list (car tree) (optimize (second tree)) (optimize (third
              tree)))))
257 ('THROW (list (car tree) (optimize (second tree)) (optimize (third
              tree)))))
258 ('APPLY (list 'APPLY (optimize (second tree)) (optimize-many (third
              tree)) (forth tree)))
259 (otherwise (comp-err "optimize-tree: invalid expression" tree))))))
260 (optimize tree)))
261
262 (defun optimize-tree2 (tree)
263   "Второй проход оптимизатора - встраивание функций, удаление после
    встраивания"
264   ;;(print `(o2 ,tree))
265   (cond ((null tree) tree)
266         ((atom tree) tree)
267         ((and (pairp tree) (cdr tree))
268          (case (car tree)
269              ('FIX-CLOSURE (if (and (cdddr tree) (forth tree) (pairp (forth tree)))
                              (list 'FIX-CLOSURE (second tree) (third tree) (if (null (forth
                              tree)) nil (list (car (forth tree)) (second tree) (optimize-tree2 (
                              third (forth tree)))))) tree))
270              ('FUNC (let* ((f (get-hash *functions-info* (second tree)))
271                             (l (list (car tree) (second tree) (optimize-tree2 (third tree))))
272                             (dc (-- (get-hash f 'decl-count))))
273                      (set-hash f 'body (third l))
274                      (set-hash f 'decl-count dc)
275                      (if (or (and (contains *optimize-flags* 'dead-code-elimination)
276                                  (or (= (get-hash f 'count) 1) (contains *optimize-flags* '
                                  all-inline)))
                          (get-hash f 'can-inline) (not (check-key f 'closure))) (> dc 0))
                          (list 'NOP) l)))
277              ('FIX-LET (let ((args (optimize-tree2 (third tree)))
278                              (body (optimize-tree2 (forth tree))))
279                          (if (and (not (search-abs-tree body '(LOCAL-SET DEEP-SET))) (
280                              one-ref-args body)
281                              (search-closure-deep-ref body) (not (side-effects args)))
                              (beta-exp body (optimize-tree2 (third tree)) 0)
                              (list 'FIX-LET (second tree) args body))))
282
283
284

```

```

285      ('FIX-CALL (beta-expansion (list 'FIX-CALL (second tree) (third tree)
      (optimize-tree2 (forth tree)))))
286      ('NARY-CALL (beta-expansion (list 'NARY-CALL (second tree) (third tree
      ) (forth tree) (optimize-tree2 (fifth tree)))))
287      (otherwise (cons (optimize-tree2 (car tree)) (optimize-tree2 (cdr tree
      ))))))
288      (t (cons (optimize-tree2 (car tree)) (optimize-tree2 (cdr tree)))))
289
290 (defun optimize-tree (tree)
291   "Оптимизация"
292   (let ((tree1 (optimize-tree1 tree)))
293     (if (contains *optimize-flags* 'beta-expansion) (optimize-tree2 tree1)
        tree1)))

```


Автор ВКР

(подпись, дата)

Д. С. Шатохин

Руководитель ВКР

(подпись, дата)

А. А. Чаплыгин

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

Место для диска